Machine Intelligence at the Edge With Learning Centric Power Allocation

Shuai Wang, *Member, IEEE*, Yik-Chung Wu, *Senior Member, IEEE*, Minghua Xia, *Member, IEEE*, Rui Wang, *Member, IEEE*, and H. Vincent Poor, *Life Fellow, IEEE*

***Abstract*— While machine-type communication (MTC) devices generate considerable amounts of data, they often cannot process the data due to limited energy and computational power. To empower MTC with intelligence, edge machine learning has been proposed. However, power allocation in this paradigm requires maximizing the learning performance instead of the communication throughput, for which the celebrated water-filling and max-min fairness algorithms become inefficient. To this end, this paper proposes learning centric power allocation (LCPA), which provides a new perspective on radio resource allocation in learning driven scenarios. By employing 1) an empirical classification error model that is supported by learning theory and 2) an uncertainty sampling method that accounts for different distributions at users, LCPA is formulated as a nonconvex nonsmooth optimization problem, and is solved using a majorization minimization (MM) framework. To get deeper insights into LCPA, asymptotic analysis shows that the transmit powers are inversely proportional to the channel gains, and scale exponentially with the learning parameters. This is in contrast to traditional power allocations where quality of wireless channels is the only consideration. Last but not least, a large-scale optimization algorithm termed mirror-prox LCPA is further proposed to enable LCPA in large-scale settings. Extensive numerical results demonstrate that the proposed LCPA algorithms outperform traditional power allocation algorithms, and the large-scale optimization algorithm reduces the computation time by orders of magnitude compared with MM-based LCPA but still achieves competing learning performance.**

***Index Terms*— Empirical classification error model, edge machine learning, learning centric communication, multiple-input multiple-output, resource allocation.**

Manuscript received June 8, 2019; revised November 9, 2019 and May 24, 2020; accepted July 12, 2020. Date of publication July 28, 2020; date of current version November 11, 2020. This work was supported in part by the National Natural Science Foundation of China under Grant 61771232 and Grant 61671488, in part by the Shenzhen Basic Research Project under Grant JCYJ20190809142403596, in part by the Natural Science Foundation of Guangdong Province under Grant 2019B1515130003, in part by the U.S. National Science Foundation under Grant CCF-0939370 and Grant CCF-1908308, in part by the Major Science and Technology Special Project of Guangdong Province under Grant 2018B010114001, and in part by the Fundamental Research Funds for the Central Universities under Grant 191gjc04. The associate editor coordinating the review of this article and approving it for publication was X. Cheng. *(Corresponding author: Yik-Chung Wu.)*

Shuai Wang is with the Department of Electrical and Electronic Engineering, Southern University of Science and Technology, Shenzhen 518055, China, and also with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: wangs3@sustech.edu.cn).

Yik-Chung Wu is with the Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong (e-mail: ycwu@eee.hku.hk).

Minghua Xia is with the School of Electronics and Information Technology, Sun Yat-sen University, Guangzhou 510006, China, and also with the Southern Marine Science and Engineering Guangdong Laboratory, Zhuhai 519082, China (e-mail: xiamingh@mail.sysu.edu.cn).

Rui Wang is with the Department of Electrical and Electronic Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: wang.r@sustech.edu.cn).

H. Vincent Poor is with the Department of Electrical Engineering, Princeton University, Princeton, NJ 08544 USA (e-mail: poor@princeton.edu).

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TWC.2020.3010522

I. Introduction

MACHINE intelligence is revolutionizing every branch of science and technology [1], [2]. If a machine wants to learn, it requires at least two ingredients: information and computation, which are usually separated from each other in machine-type communication (MTC) systems [3]. Nonetheless, sending vast volumes of data from MTC devices to the cloud not only leads to a heavy communication burden but also increases the transmission latency. To address this challenge brought by MTC, a promising solution is the *edge machine learning* technique [4]–[11] that trains a machine learning model or *fine-tunes a pre-trained model* at the edge, i.e., at a nearby radio access point with computation resources.

In general, there are two ways to implement edge machine learning: data sharing and model sharing. Data sharing uses the edge to collect data generated from MTC devices for machine learning [4]–[7], while model sharing uses federated learning [8]–[11] to exchange model parameters (instead of data) between the edge and users. Both approaches are recognized as key paradigms in the sixth generation (6G) wireless communications [12]–[14]. However, since the MTC devices often cannot process the data due to limited computational power, this paper focuses on data sharing.

A. Motivation and Related Work

In contrast to conventional communication systems, edge machine learning systems aim to maximize the learning performance instead of the communication throughput. Therefore, edge resource allocation becomes very different from traditional resource allocation schemes that merely consider the wireless channel state information [15]–[18]. For instance, the celebrated water-filling scheme allocates more resources to better channels for throughput maximization [15], and the max-min fairness scheme allocates more resources to cell-edge users to maintain certain quality of service [16]. While these two schemes have proven to be very efficient in traditional wireless communication systems, they could lead to poor

1536-1276 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

learning performance in edge learning systems because they do not account for the machine learning factors such as model and dataset complexities. Imagine training a deep neural network (DNN) and a support vector machine (SVM) at the edge. Due to much larger number of parameters in DNN, the edge should allocate more resources to MTC devices that upload data for the DNN than those for the SVM.

Nonetheless, in order to maximize the learning performance, we need a mathematical expression of the learning performance with respect to the number of samples, which does not exist to the best of the authors' knowledge. While the sample complexity of a learning task can be related to the Vapnik-Chervonenkis (VC) dimension [1], this theory only provides a vague estimate that is independent of the specific learning algorithm or data distribution. To better understand the learning performance, it has been proved in [19], [20] that the generalization error can be upper bounded by the summation of the bias between the model's prediction and the optimal prediction, the variance due to training datasets, and the noise of the target example. With the bound being tight for certain loss functions (e.g., squared loss and zero-one loss), the bias-variance decomposition theory gives rise to an empirical nonlinear classification error model [21]–[23] that is also theoretically supported by the inverse power law derived via statistical mechanics [24].

B. Summary of Results

In this paper, we adopt the above nonlinear model to approximate the learning performance, and a *learning centric power allocation (LCPA)* problem is formulated with the aim of minimizing classification error subject to the total power budget constraint. When the data is non-independent-and-identically-distributed (non-IID) among users, the LCPA problem can also be flexibly integrated with uncertainty sampling. Since the formulated machine learning resource allocation problem is nonconvex and nonsmooth, it is nontrivial to solve. By leveraging the majorization minimization (MM) framework from optimization, an MM-based LCPA algorithm that obtains a Karush-Kuhn-Tucker (KKT) solution is proposed. To get deeper insights into LCPA, asymptotic analysis with the number of antennas at the edge going to infinity is provided. The asymptotic optimal solution discloses that the transmit powers are inversely proportional to the channel gain, and scale exponentially with the classification error model parameters. This result reveals that machine learning has a stronger impact than wireless channels in LCPA. To enable affordable computational complexity when the number of MTC devices is large, a variant of LCPA, called mirror-prox LCPA, is proposed. The algorithm is a first-order method (FOM), implying that its complexity is linear with respect to the number of users. Extensive experimental results based on public datasets show that the proposed LCPA scheme is able to achieve a higher classification accuracy than that of the sum-rate maximization and max-min fairness power allocation schemes. For the first time, the benefit brought by joint communication and learning design is quantitatively demonstrated in edge machine learning systems. Our results also show that the mirror-prox LCPA reduces the computation time by orders of magnitude compared to the MM-based LCPA but still achieves satisfactory performance.

To sum up, the contributions of this paper are listed as follows.

- The LCPA scheme is developed for the edge machine learning problem, which maximizes the learning accuracy instead of the communication throughput.
- To understand how LCPA works, an asymptotic optimal solution to the edge machine learning problem is derived, which, for the first time, discloses that the transmit power obtained from LCPA grows linearly with the path-loss and grows exponentially with the learning parameters.
- To reduce the computation time of LCPA in the massive multiple-input multiple-output (MIMO) setting, a variant of LCPA based on FOM is proposed, which enables the edge machine learning system to scale up the number of MTC users.
- Extensive experimental results based on public datasets (e.g., MNIST, CIFAR-10, ModelNet40) show that the proposed LCPA is able to achieve a higher accuracy than that of the sum-rate maximization and max-min fairness schemes.

C. Outline

The rest of this paper is organized as follows. System model and problem formulation are described in Section II. Classification error modeling is presented in Section III. The MM-based LCPA algorithm, the asymptotic analysis, and the large-scale optimization method are derived in Sections IV, V, and VI, respectively. Finally, experimental results are presented in Section VII, and conclusions are drawn in Section VIII.

Notation: Italic letters, lowercase and uppercase bold letters represent scalars, vectors, and matrices, respectively. Curlicue letters stand for sets and $|\cdot|$ is the cardinality of a set. The operators $(\cdot)^T$, $(\cdot)^H$ and $(\cdot)^{-1}$ take the transpose, Hermitian and inverse of a matrix, respectively. We use $(a_1, a_2, \cdots)$ to represent a sequence, $[a_1, a_2, \cdots]^T$ to represent a column vector, and $\|\cdot\|_p$ to represent the ℓ_p-norm of a vector. The symbol $\mathbf{I}_N$ indicates the $N \times N$ identity matrix, $\mathbf{1}_N$ indicates the $N \times 1$ vector with all entries being unity, and $\mathcal{CN}(0,1)$ stands for complex Gaussian distribution with zero mean and unit variance. The function $[x]^+ = \max(x, 0)$, while $\exp(\cdot)$ and $\ln(\cdot)$ denote the exponential function and the natural logarithm function, respectively. The function $\lfloor x \rfloor = \max\{n \in \mathbb{Z} : n \leq x\}$. Finally, $\mathbb{E}(\cdot)$ means the expectation of a random variable, $\mathbb{I}_{\mathcal{A}}(x) = 1$ if $x \in \mathcal{A}$ and zero otherwise, and $\mathcal{O}(\cdot)$ means the order of arithmetic operations. For ease of retrieval, important variables and parameters to be used in this paper are listed in Table I.

II. System Model and Problem Formulation

We consider an edge machine learning system shown in Fig. 1, which consists of an intelligent edge with N antennas and K users with datasets $\{\mathcal{D}_1, \cdots, \mathcal{D}_K\}$. The goal of the edge is to train M classification models by collecting

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

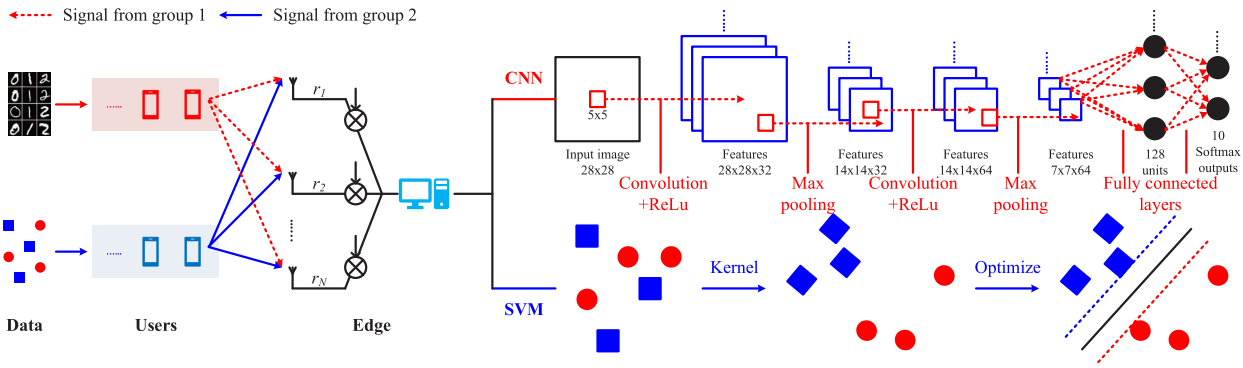


Fig. 1. System model of machine intelligence at the edge.

TABLE I

SUMMARY OF IMPORTANT VARIABLES AND PARAMETERS

Symbol	Type	Description
$p_k \in \mathbb{R}_+$	Variable	Transmit power (in Watt) at user k.
$v_m \in \mathbb{Z}_+$	Variable	Number of training samples for task m.
P	Parameter	Total transmit power budget (in Watt).
B	Parameter	Communication bandwidth (in Hz).
T	Parameter	Transmission time (in s).
ξ	Parameter	The factor accounting for packet loss and network overhead.
σ^2	Parameter	Noise power (in Watt).
D_m	Parameter	Data size (in bit) per sample for task m
A_m	Parameter	Number of historical samples at the edge for task m.
$G_{k,l}$	Parameter	The composite channel gain from user l to the edge when decoding data of user k.
$\mathcal{D}_k, \mathcal{H}_k, \mathcal{T}_k, \mathcal{V}_k$	Dataset	The full dataset, historical dataset, training dataset, validation dataset of user k.
$\Psi_m(v_m)$	Function	Classification error of the learning model m when the sample size is v_m.
$\Theta_m(v_m \vert a_m, b_m)$	Function	Empirical classification error model for task m with parameters (a_m, b_m).
$U(\mathbf{d})$	Function	Prediction confidence of sample $\mathbf{d}$.

data from M user groups (e.g., UAVs with camera sensors) $\{\mathcal{Y}_1, \mathcal{Y}_2, \cdots, \mathcal{Y}_M\}$, with the group $\mathcal{Y}_m$ containing all users having data for training the model m. In case where some data from a particular user is used to train both model m and model j, we can allow $\mathcal{Y}_m$ and $\mathcal{Y}_j$ to include a common user, i.e., $\mathcal{Y}_m \bigcap \mathcal{Y}_j \neq \emptyset$ for $m \neq j$. For the classification models, without loss of generality, Fig. 1 depicts a convolutional neural network (CNN) and a support vector machine (SVM) (i.e., $M = 2$), but more user groups and other classification models are equally valid. It is assumed that the data are labeled at the edge. This can be supplemented by the recent self-labeled techniques [25], [26], where a classifier is trained with an initial small number of labeled examples, and then the model is retrained with its own most confident predictions, thus enlarging its labeled training set. After training the classifiers, the edge can feedback the trained models to users for subsequent use (e.g., object detection). Notice that if the classifiers are pre-trained at the cloud and deployed at the edge, the task of edge machine learning is to fine-tune the pre-trained models at the edge, using local data generated from MTC users.

More specifically, the user $k \in \{1, \cdots, K\}$ transmits a signal s_k with power $\mathbb{E}[|s_k|^2] = p_k$. Accordingly, the received signal $\mathbf{r} = [r_1, \cdots, r_N]^T \in \mathbb{C}^{N\times 1}$ at the edge is $\mathbf{r} = \sum_{k=1}^{K} \mathbf{h}_k s_k + \mathbf{n}$, where $\mathbf{h}_k \in \mathbb{C}^{N\times 1}$ is the channel vector from the kth user to the edge, and $\mathbf{n} \sim \mathcal{CN}(\mathbf{0}, \sigma^2 \mathbf{I}_N)$. By applying the maximal ratio combining (MRC) receiver $\mathbf{h}_k / \|\mathbf{h}_k\|_2$ to $\mathbf{r}$, the data-rate of user k is

$$R_k = \log_2 \left(1 + \frac{G_{k,k} p_k}{\sum_{l=1, l\neq k}^{K} G_{k,l} p_l + \sigma^2} \right), \tag{1}$$

where $G_{k,l}$ represents the composite channel gain (including channel fading and MIMO processing) from user l to the edge when decoding data of user k:

$$G_{k,l} = \begin{cases} \|\mathbf{h}_k\|_2^2, & \text{if } k = l \\ \dfrac{|\mathbf{h}_k^H \mathbf{h}_l|^2}{\|\mathbf{h}_k\|_2^2}, & \text{if } k \neq l. \end{cases} \tag{2}$$

With the expression of R_k in (1), the amount of data in bit received from user k is BTR_k, where constant B is the bandwidth in Hz that is assigned to the system (e.g., a standard MTC system would have 180 kHz bandwidth [27]), and T is the total number of transmission time in second. As a result, the total number of training samples that are collected at the edge for training the model m is

$$v_m = \xi \sum_{k\in\mathcal{Y}_m} \left\lfloor \frac{BTR_k}{D_m} \right\rfloor + A_m \approx \sum_{k\in\mathcal{Y}_m} \frac{\xi BTR_k}{D_m} + A_m, \tag{3}$$

where A_m is the number of historical samples for task m residing at the edge, D_m is the number of bits for each data sample, and $\xi \leq 1$ is a factor accounting for the reduced number of samples due to packet loss and network overhead. The approximation is due to $\lfloor x \rfloor \to x$ when $x \gg 1$.

For example, in real-world edge/cloud machine learning applications, the data needs to be uploaded multiple times [28], [29] (e.g., twice a day). If the historical dataset of user k at the edge is $\mathcal{H}_k$, then $A_m = \sum_{k\in\mathcal{Y}_m} |\mathcal{H}_k|$. For D_m, if we consider the MNIST dataset [30], since the handwritten digit images are gray scale with 28×28 pixels (each pixel has 8 bits), in this case $D_m = 8 \times 28 \times 28 + 4 = 6276$ bits

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

(4 bits are reserved for the labels of 10 classes [30] in case the users also transmit labels). Lastly, if the system reserves 30% of the resource blocks for network overhead [31], and the probability of packet error rate is 0.1 [32], then ξ can be estimated as $\xi = 0.7 \times 0.9 = 0.63$.

In the considered system, the design variables that can be controlled are the transmit powers of different users $\mathbf{p} = [p_1, \cdots, p_K]^T$ and the sample sizes of different models $\mathbf{v} = [v_1, \cdots, v_M]^T$. Since the power costs at users should not exceed the total budget P, the variable $\mathbf{p}$ needs to satisfy $\sum_{k=1}^{K} p_k \leq P$. Since a larger $\sum_{k=1}^{K} p_k$ always improves the learning performance, we can rewrite $\sum_{k=1}^{K} p_k \leq P$ as $\sum_{k=1}^{K} p_k = P$. Having the transmit power satisfied, it is then crucial to minimize the classification errors (i.e., the number of incorrect predictions divided by the number of total predictions), which leads to the following learning centric power allocation (LCPA) problem:

$$\mathrm{P}: \min_{\mathbf{p},\mathbf{v}} \max_{m=1,\cdots,M} \Psi_m(v_m),$$

$$\text{s.t.} \sum_{k=1}^{K} p_k = P, \quad p_k \geq 0, \quad k = 1, \cdots, K, \tag{4a}$$

$$\sum_{k \in \mathcal{Y}_m} \frac{\xi BT}{D_m} \log_2 \left(1 + \frac{G_{k,k} p_k}{\sum_{l=1, l \neq k}^{K} G_{k,l} p_l + \sigma^2}\right) + A_m = v_m, \quad m = 1, \cdots, M, \tag{4b}$$

where $\Psi_m(v_m)$ is the classification error of the learning model m when the sample size is v_m, and the min-max operation at the objective function is to guarantee the worst-case learning performance. The key challenge to solve P is that functions $(\Psi_1, \cdots, \Psi_M)$ represent generalization errors, and to the best of the authors' knowledge, currently there is no exact expression of $\Psi_m(v_m)$. To address this issue, Section III will adopt an empirical classification error model to approximate Ψ_m.

The problem formulation P has assumed that the data is IID among different users (i.e., different users have identical distributions). However, practical applications may involve non-IID data distributions. For example, user 1 has 1000 samples but they are exactly the same (e.g., repeating the first sample in the MNIST dataset); and user 2 has 50 different samples (e.g., randomly drawn from the MNIST dataset). In this case, upper bound and lower bound data-rate constraints should be imposed depending on the quality of users' data (i.e., whether their data improves the learning performance and how much the improvement could be):

- If the data transmitted from user k does not help to improve the learning performance, a maximum amount of transmitted data $Z_k^{\max}$ should be imposed. This is the case of user 1, since the well-understood data should not be transmitted to the edge over and over again.
- On the other hand, if the data transmitted from user k helps to improve the learning performance, a minimum amount of data $Z_k^{\min}$ should be imposed on this user. This is the case of user 2, since more data from this user would reduce the learning error.

Notice that both the lower and upper bounds can be imposed on the same user simultaneously. Therefore, we can add the following constraint to problem P:

$$Z_k^{\min} \leq \frac{\xi BT}{D_m} \log_2 \left(1 + \frac{G_{k,k} p_k}{\sum_{l=1, l \neq k}^{K} G_{k,l} p_l + \sigma^2}\right) \leq Z_k^{\max}, \quad k = 1, \cdots, K. \tag{5}$$

Of course, the remaining question is how to determine whether the data from a particular user is useful or not. To this end, the uncertainty sampling method [33] can be adopted. In particular, let the function $\mathbb{P}(\mathbf{c}|\mathbf{d}, \mathbf{w})$ denote the probability of the label being $\mathbf{c}$ given sample $\mathbf{d} \in \mathbb{R}^{S \times 1}$ and the model parameter vector $\mathbf{w}$ (e.g., $\mathbf{c}$ is a one-hot vector containing 10 elements if the learning problem is a 10-classes classification; for the MNIST dataset, we have $S = 784$; and in the considered CNN model, $\mathbf{w}$ contains the convolution matrices and the bias vectors). The confidence of the predicted label $\mathbf{c}^* = \text{argmax}_{\mathbf{c}} \mathbb{P}(\mathbf{c}|\mathbf{d}, \mathbf{w})$ is

$$U(\mathbf{d}) = \mathbb{P}(\mathbf{c}^*|\mathbf{d}, \mathbf{w}). \tag{6}$$

Therefore, we can compute the $U(\mathbf{d})$ for all the historical samples from user k. If the average (or median, minimum) value of $U(\mathbf{d})$ is large (e.g., > 0.9) for the historical data from user k, then user k should have its data-rate upper bounded. On the other hand, if the average (or median, minimum) value of $U(\mathbf{d})$ is small (e.g., < 0.5), then user k should have its data-rate lower bounded. The entire LCPA scheme with uncertainty sampling is summarized in Fig. 2.

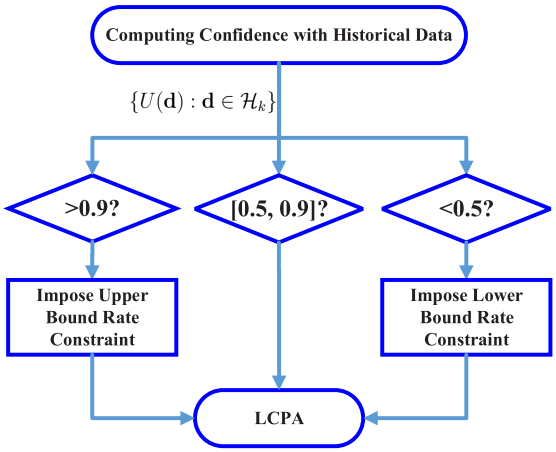


Fig. 2. Illustration of the LCPA scheme with uncertainty sampling.

Remark 1 (Practical Power Control Procedure): The LCPA problem will be solved at the edge, which can then inform the users about their transmit powers through downlink control channels. For example, the 3GPP standard reserves some resource blocks for Physical Downlink Control Channel (PDCCH) [31], which are used for sending control signals such as users' transmit powers and modulation orders.

III. Modeling of Classification Error

A. Classification Error Rate Function

In general, the classification error $\Psi_m(v_m)$ is a nonlinear function of v_m [19]–[24]. Particularly, this nonlinear function should satisfy the following properties:

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

(i) As Ψ_m is a percentage, it satisfies $0 \leq \Psi_m(v_m) \leq 1$;
(ii) Since more data would provide more information, $\Psi_m(v_m)$ is a monotonically decreasing function of v_m [21];
(iii) As v_m increases, the magnitude of the partial derivative $|\partial \Psi_m / \partial v_m|$ would gradually decrease and become zero when v_m is sufficiently large [22], meaning that increasing sample size no longer helps machine learning.

Based on the properties (i)–(iii), the following nonlinear model $\Theta_m(v_m|a_m, b_m)$ [21]–[23] can be used to capture the shape of $\Psi_m(v_m)$:

$$\Psi_m(v_m) \approx \Theta_m(v_m|a_m, b_m) = a_m \, v_m^{-b_m}, \tag{7}$$

where $a_m, b_m \geq 0$ are tuning parameters. It can be seen that Θ_m satisfies all the above properties. Moreover, $\Theta_m(v_m|a_m, b_m) \to 0$ if $v_m \to +\infty$, meaning that the error is 0 with infinite data.[1]

Interpretation From Learning Theory: Apart from (i)–(iii), the model (7) corroborates the *inverse power relationship* between learning performance Ψ_m and the amount of training data v_m from the perspective of statistical mechanics [24]. In particular, according to [24], the training procedure can be modeled as a Gibbs distribution of networks characterized by a temperature parameter T_g. The asymptotic generalization error as the number of samples v_m goes to infinity is expressed as [24, Eq. (3.12)]

$$\Psi_m(v_m) \to \epsilon_{\min} + \left(\frac{T_g}{2} + \frac{\mathrm{Tr}(\mathbf{U}_m \mathbf{V}_m^{-1})}{2W_m} \right) W_m v_m^{-1}, \quad \text{if } v_m \to +\infty, \tag{8}$$

where $\epsilon_{\min} \geq 0$ is the minimum error for all possible learning models and W_m is the number of parameters. The matrices $(\mathbf{U}_m, \mathbf{V}_m)$ contain the second-order and first-order derivatives of the generalization error function with respect to the parameters of model m. By comparing (8) with (7), we can see that $a_m = \left(\frac{T_g}{2} + \frac{\mathrm{Tr}(\mathbf{U}_m \mathbf{V}_m^{-1})}{2W_m} \right) W_m$ and $b_m = -1$ in the asymptotic case. Therefore, as the number of samples goes to infinity, the weighting factor a_m in (7) accounts for the *model complexity* of the classifier m. Moreover, in the finite sample size regime, the slopes of learning curves may be different for different datasets even for the same machine learning model. This means that v_m^{-1} is not always suitable in practice. Therefore, b_m is introduced as a tuning parameter accounting for the correlation in a dataset.

B. Parameter Fitting of CNN and SVM Classifiers

We use the public MNIST dataset [30] as the input images, and train the 6-layer CCN (shown in Fig. 1) with training sample size $v_m^{(i)}$ ranging from 100 to 10000. In particular, the input image is sequentially fed into a 5×5 convolution layer (with ReLu activation, 32 channels, and SAME padding), a 2×2 max pooling layer, then another 5×5 convolution layer (with ReLu activation, 64 channels, and SAME padding), a 2×2 max pooling layer, a fully connected layer with 128 units (with ReLu activation), and a final softmax output layer (with 10 ouputs). The training procedure is implemented via Adam optimizer with a learning rate of 10^{-4} and a mini-batch size of 100. After training for 5000 iterations, we test the trained model on a dataset with 1000 unseen samples, and compute the corresponding classification error. By varying the sample size v_m as $(v_m^{(1)}, v_m^{(2)}, \cdots) = (100, 150, 200, 300, 500, 1000, 5000, 10000)$, we can obtain the classification error $\Psi_m(v_m^{(i)})$ for each sample size $v_m^{(i)}$, where $i = 1, \cdots, Q$, and $Q = 8$ is the number of points to be fitted. With $\{v_m^{(i)}, \Psi_m(v_m^{(i)})\}_{i=1}^{Q}$, the parameters (a_m, b_m) in Θ_m can be found via the following nonlinear least squares fitting:

$$\min_{a_m, \, b_m} \ \frac{1}{Q} \sum_{i=1}^{Q} \left| \Psi_m \left(v_m^{(i)} \right) - \Theta_m \left(v_m^{(i)} | a_m, b_m \right) \right|^2, \quad \text{s.t.} \quad a_m \geq 0, \quad b_m \geq 0. \tag{9}$$

The above problem can be solved by two-dimensional brute-force search, or gradient descent method. Since the parameters (a_m, b_m) for different tasks are obtained independently, the total complexity is linear in terms of the number of tasks. The fitted classification error versus the sample size is shown in Fig. 3a. It is observed from Fig. 3a that with the parameters $(a_m, b_m) = (9.27, 0.74)$, the nonlinear classification error model in (7) matches the experimental data of CNN very well.

To demonstrate the versatility of the model, we also fit the nonlinear model to the classification error of a support vector machine (SVM) classifier. The SVM uses penalty coefficient $C = 1$ and Gaussian kernel function $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\widetilde{\gamma} \, \|\mathbf{x}_i - \mathbf{x}_j\|_2^2 \right)$ with $\widetilde{\gamma} = 0.001$ [34]. Moreover, the SVM classifier is trained on the digits dataset in the Scikit-learn Python machine learning tookbox, and the dataset contains 1797 images of size 8×8 from 10 classes, with 5 bits (corresponding to integers 0 to 16) for each pixel [34]. Therefore, each image needs $D_m = 8 \times 8 \times 5 + 4 = 324$ bits. Out of all images, we train the SVM using the first 1000 samples with sample size $(v_m^{(1)}, v_m^{(2)}, \cdots) = (30, 50, 100, 200, 300, 400, 500, 1000)$, and use the latter 797 samples for testing. The parameters (a_m, b_m) for the SVM are obtained following a similar procedure in (9). It is observed from Fig. 3a that with $(a_m, b_m) = (6.94, 0.8)$, the model in (7) fits the experimental data of SVM.

C. Practical Implementation

One may wonder how could one obtain the fitted classification error model before the actual machine learning model is being trained. There are two ways to address this issue.

1) Extrapolation: More specifically, the error function can be obtained by training the machine learning model on the historical dataset at the edge, and the performance on a future larger dataset can be predicted. This is called extrapolation [21]. For example, by fitting the error function to the first half experimental data of CNN in Fig. 3b (i.e., $v_m = (100, 150, 200, 300)$), we can obtain $(a_m, b_m) = (7.3, 0.69)$, and the resultant curve predicts the errors at $v_m = (500, 1000, 5000, 10000)$ very well as shown in Fig. 3b.

[1]We assume the model is powerful enough such that given infinite amount of data, the error rate can be driven to zero.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

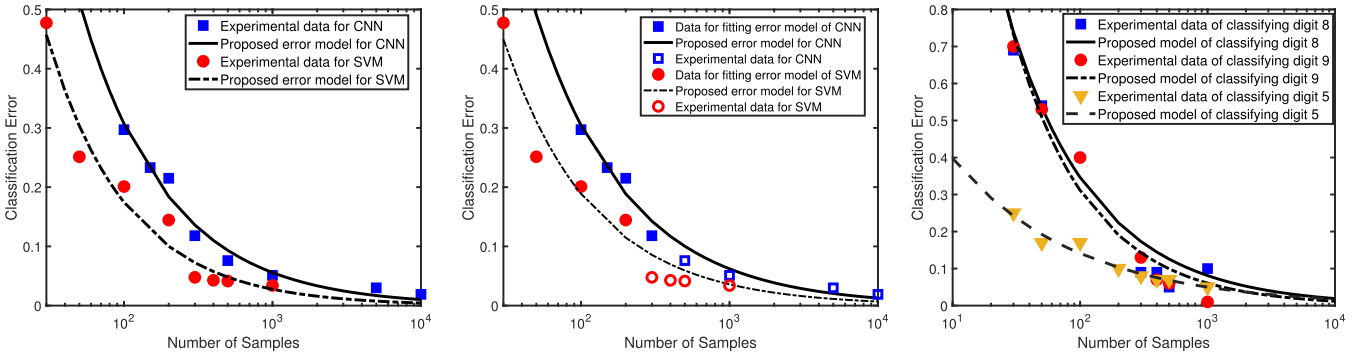


Fig. 3. a) Comparison between the experimental data and the nonlinear classification error model. The parameters in the models are given by $(a_m, b_m) = (9.27, 0.74)$ for CNN and $(a_m, b_m) = (6.94, 0.8)$ for SVM; b) Fitting the error function to historical datasets. The parameters in the models are given by $(a_m, b_m) = (7.3, 0.69)$ for CNN and $(a_m, b_m) = (5.2, 0.72)$ for SVM; c) Comparison between different classification tasks.



Fig. 4. a) Comparison between the experimental data and the nonlinear classification error model. The parameters in the models are given by $(a_m, b_m) = (8.15, 0.44)$ for ResNet-110 and $(a_m, b_m) = (0.96, 0.24)$ for PointNet; b) Examples of the 3D point clouds in ModelNet40; c) The flowchart for LCPA Algorithms.

Similarly, with $(a_m, b_m) = (5.2, 0.72)$ obtained from the experimental data of $v_m = (30, 50, 100, 200)$, the proposed model for SVM matches the classification errors at $v_m = (300, 400, 500, 1000)$. It can be seen that the fitting performance in Fig. 3b is slightly worse than that in Fig. 3a, as we use smaller number of pilot data. But since our goal is to distinguish different tasks rather than accurate prediction of the classification errors, the extrapolation method can guide the resource allocation at the edge.

2) Approximation: This means that we can pre-train a large number of commonly-used models offline (not at the edge) and store their corresponding parameters of (a_m, b_m) in a look-up table at the edge. Then by choosing a set of parameters from the table, the unknown error model at the edge can be approximated. This is because the error functions can share the same trend for two similar tasks, e.g., classifying digit '8' and '9' with SVM as shown in Fig. 3c. Notice that there may be a mismatch between the pre-training task and the real task at the edge. This is the case between classifying digit '8' and '5' in Fig. 3c. As a result, it is necessary to carefully measure the similarity between two tasks when choosing the parameters.

D. Parameter Fitting of ResNet and PointNet

To verify the nonlinear model in (7) under deeper learning models and larger datasets, we train the 110-layer deep residual network (ResNet-110 with 1.7 M parameters) [35] using the CIFAR-10 dataset as the input images, with training sample size ranging from 5000 to 50000. The image in the CIFAR-10 dataset has 32×32 pixels (each pixel has 3 Bytes representing RGB), and each image sample has a size of $(32 \times 32 \times 3 + 1) \times 8 = 24584$ bits. The training procedure is implemented with a diminishing learning rate and a mini-batch size of 100. After training for 50000 iterations (~ 2.5 hours), we test the trained model on a dataset with 10000 unseen samples, and obtain the corresponding classification error. It can be seen from Fig. 4a that the proposed model with $(a_m, b_m) = (8.15, 0.44)$ matches the experimental data of ResNet-110 very well. Moreover, we also consider the PointNet (3.5 M parameters), which applies feature transformations and aggregates point features by max pooling [36, Fig. 2] to classify 3D point clouds dataset ModelNet40 (see examples in Fig. 4b). In ModelNet40, there are 12311 CAD models from 40 object categories, split into 9843 for training and 2468 for testing. Each sample has 2000 points with three single-precision floating-point coordinates (4 Bytes), and the data size per sample is $(2000 \times 3 \times 4 + 1) \times 8 = 192008$ bits. After training for 250 epochs (~ 5.5 hours) with a mini-batch of 32, the classification error versus the number of samples is obtained in Fig. 4a, and the proposed classification error model with $(a_m, b_m) = (0.96, 0.24)$ matches the experimental data of PointNet very well.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

IV. MM-Based LCPA Algorithm

Based on the results in Section III, we can directly approximate the true error function Ψ_m by Θ_m. However, to account for the approximation error between Ψ_m and Θ_m (e.g., due to noise in samples or slight mismatch between data used for fitting and data observed in MTC devices), a weighting factor $\beta_m \geq 1$ can be applied to Θ_m, where a higher value of β_m accounts for a larger approximation error.[2] Then by replacing Ψ_m with $\beta_m \Theta_m$ and putting (4b) into $\Theta_m(v_m|a_m, b_m)$ to eliminate $\mathbf{v}$, problem P becomes:

$$\begin{aligned}
\mathrm{P1}: \ \min_{\mathbf{p}} \quad & \max_{m=1,\cdots,M} \ \beta_m \, \Phi_m(\mathbf{p}), \\
\text{s.t.} \quad & \sum_{k=1}^{K} p_k = P, \quad p_k \geq 0, \quad \forall k, && \text{(10a)} \\
& G_{k,k} p_k \geq \left(2^{D_m Z_k^{\min}/(\xi BT)} - 1\right) \\
& \quad \times \left(\sum_{l \neq k} G_{k,l} p_l + \sigma^2\right), \quad \forall k, && \text{(10b)} \\
& G_{k,k} p_k \leq \left(2^{D_m Z_k^{\max}/(\xi BT)} - 1\right) \\
& \quad \times \left(\sum_{l \neq k} G_{k,l} p_l + \sigma^2\right), \quad \forall k, && \text{(10c)}
\end{aligned}$$

where constraints (10b)–(10c) come from equation (5) in the non-IID case and

$$\Phi_m(\mathbf{p}) := a_m \left[\sum_{k \in \mathcal{Y}_m} \frac{\xi BT}{D_m} \log_2 \left(1 + \frac{G_{k,k} p_k}{\sum_{l \neq k} G_{k,l} p_l + \sigma^2}\right) + A_m\right]^{-b_m}. \tag{11}$$

It can be seen that P1 is a nonlinear optimization problem due to the nonlinear classification error model (7). Moreover, the $\max$ operator introduces non-smoothness to the problem, and the objective function is not differentiable. Thus the existing method based on gradient descent [18] is not applicable.

To solve P1, we propose to use the framework of MM [37]–[40], which constructs a sequence of upper bounds $\{\widetilde{\Phi}_m\}$ on $\{\Phi_m\}$ and replaces $\{\Phi_m\}$ in P1 with $\{\widetilde{\Phi}_m\}$ to obtain the surrogate problems. More specifically, given any feasible solution $\mathbf{p}^\star$ to P1, we define surrogate functions

$$\begin{aligned}
&\widetilde{\Phi}_m(\mathbf{p}|\mathbf{p}^\star) \\
&= a_m \Bigg\{ \sum_{k \in \mathcal{Y}_m} \frac{\xi BT}{D_m \ln 2} \Bigg[\ln \left(\sum_{l=1}^{K} \frac{G_{k,l} p_l}{\sigma^2} + 1\right) \\
&\quad - \ln \left(\sum_{l=1, l \neq k}^{K} \frac{G_{k,l} p_l^\star}{\sigma^2} + 1\right) - \left(\sum_{l=1, l \neq k}^{K} \frac{G_{k,l} p_l^\star}{\sigma^2} + 1\right)^{-1} \\
&\quad \times \left(\sum_{l=1, l \neq k}^{K} \frac{G_{k,l} p_l}{\sigma^2} + 1\right) + 1 \Bigg] + A_m \Bigg\}^{-b_m}, \qquad \text{(12)}
\end{aligned}$$

and the following proposition can be established.

Proposition 1: The functions $\{\widetilde{\Phi}_m\}$ satisfy the following conditions:

(i) Upper bound condition: $\widetilde{\Phi}_m(\mathbf{p}|\mathbf{p}^\star) \geq \Phi_m(\mathbf{p})$.

(ii) Convexity: $\widetilde{\Phi}_m(\mathbf{p}|\mathbf{p}^\star)$ is convex in $\mathbf{p}$.

(iii) Local equality condition: $\widetilde{\Phi}_m(\mathbf{p}^\star|\mathbf{p}^\star) = \Phi_m(\mathbf{p}^\star)$ and $\nabla_{\mathbf{p}} \widetilde{\Phi}_m(\mathbf{p}^\star|\mathbf{p}^\star) = \nabla_{\mathbf{p}} \Phi_m(\mathbf{p}^\star)$.

Proof: See Appendix A. ■

With part (i) of **Proposition 1**, an upper bound can be directly obtained if we replace the functions $\{\Phi_m\}$ by $\{\widetilde{\Phi}_m\}$ around a feasible point. However, a tighter upper bound can be achieved if we treat the obtained solution as another feasible point and continue to construct the next-round surrogate function. In particular, assuming that the solution at the nth iteration is given by $\mathbf{p}^{[n]}$, the following problem is considered at the $(n+1)^{\text{th}}$ iteration:

$$\begin{aligned}
\mathrm{P1}[n+1]: \ \min_{\mathbf{p}} \quad & \max_{m=1,\cdots,M} \ \beta_m \widetilde{\Phi}_m(\mathbf{p}|\mathbf{p}^{[n]}), \\
\text{s.t.} \quad & \text{constraints (10a)} - \text{(10c)}. \qquad \text{(13)}
\end{aligned}$$

Based on part (ii) of **Proposition 1**, the problem $\mathrm{P1}[n+1]$ is convex and can be solved by off-the-shelf software packages (e.g., CVX Mosek [41]) for convex programming. Denoting its optimal solution as $\mathbf{p}^*$, we can set $\mathbf{p}^{[n+1]} = \mathbf{p}^*$, and the process repeats with solving the problem $\mathrm{P1}[n+2]$. According to part (iii) of **Proposition 1** and [37, Theorem 1], every limit point of the sequence $(\mathbf{p}^{[0]}, \mathbf{p}^{[1]}, \cdots)$ is a KKT solution to P1 as long as the starting point $\mathbf{p}^{[0]}$ is feasible to P1 (e.g., $\mathbf{p}^{[0]} = P/K \, \mathbf{1}_K$). The entire procedure of the MM-based LCPA is summarized in the left hand branch of Fig. 4c.

In terms of computational complexity, $\mathrm{P1}[n+1]$ involves K primal variables and $M + 3K + 1$ dual variables. The dual variables correspond to $M + 3K + 1$ constraints in $\mathrm{P1}[n+1]$, where M constraints come from the $\max$ operator, K constraints come from nonnegative power constraints, $2K$ constraints come from the data-rate bounds, and one constraint comes from the power budget. Therefore, the worst-case complexity for solving $\mathrm{P1}[n+1]$ is $\mathcal{O}((M + 4K + 1)^{3.5})$ [42]. Consequently, the total complexity for solving P1 is $\mathcal{O}(\mathcal{I}\,(M+4K+1)^{3.5})$, where $\mathcal{I}$ is the number of iterations needed for the algorithm to converge.

V. Asymptotic Analysis and Insights to LCPA

To understand how LCPA works, this section investigates the asymptotic case when the number of antennas at the edge approaches infinity (i.e., $N \to +\infty$). Moreover, we consider the special case of $|\mathcal{Y}_m| = 1$ and $\mathcal{Y}_m \bigcap \mathcal{Y}_j = \emptyset$ (i.e., each user group has only one unique user). For notational simplicity, we denote the unique user in group $\mathcal{Y}_m$ as user $m = k$. As each task only involves one user, non-IID data distribution among users in a single task does not exist, and therefore constraints (10b)–(10c) can be removed. On the other hand, as $N \to +\infty$, the channels from different users to the edge would be asymptotically orthogonal [43]–[45] and we have

$$G_{k,l} = \frac{|\mathbf{h}_k^H \mathbf{h}_l|^2}{\|\mathbf{h}_k\|_2^2} \to 0, \quad \forall k \neq l. \tag{14}$$

[2]Since the real classification error is scattered around the fitted one, introducing $\rho_m \geq 1$ means that we need to elevate the fitted curves to account for the possibilities that worse classification results may happen compared to our prediction. In other words, we should be more conservative (pessimistic) about our prediction.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

Based on such orthogonality feature, and putting $G_{k,l}=0$ for $k\neq l$ into Φ_m in P1, problem P1 under $N\to+\infty$ and $|\mathcal{Y}_m|=1$ is rewritten as

$$\begin{aligned}\mathrm{P2}:\ \min_{\mathbf{p},\,\mu}\quad & \mu,\\ \text{s.t. }& \beta_k a_k\left(\frac{\xi BT}{D_k}\log_2\left(1+\frac{G_{k,k}p_k}{\sigma^2}\right)+A_k\right)^{-b_k}\leq\mu,\quad \forall k,\\ & \sum_{k=1}^{K}p_k=P,\quad p_k\geq 0,\quad \forall k,\end{aligned}\tag{15}$$

where $\mu\in[0,1]$ is a slack variable and has the interpretation of *classification error level*. The following proposition gives the optimal solution to P2.

Proposition 2: The optimal $\mathbf{p}^*$ *to* P2 *is*

$$p_k^*(\mu)=\left[\frac{\sigma^2}{G_{k,k}}\exp\left(\frac{D_k\ln 2}{\xi BT}\left[\left(\frac{\mu}{\beta_k a_k}\right)^{-1/b_k}-A_k\right]\right)-\frac{\sigma^2}{G_{k,k}}\right]^+,\quad k=1,\cdots,K,\tag{16}$$

where μ *satisfies* $\sum_{k=1}^{K}p_k^*(\mu)=P$.

Proof: See Appendix B. ∎

A. Computing the Classification Error Level

To efficiently compute the classification error level μ, it is observed that the function $p_k^*(\mu)$ is a decreasing function of μ. Therefore, the classification error level μ can be obtained from solving $\sum_{k=1}^{K}p_k^*(\mu)=P$ using bisection method within interval $[0,1]$. More specifically, given $\mu_{\max}$ and $\mu_{\min}$ (initially $\mu_{\max}=1$ and $\mu_{\min}=0$), we set $\mu=(\mu_{\max}+\mu_{\min})/2$. If $\sum_k p_k^*(\mu)\geq P$, we update $\mu_{\min}=\mu$; otherwise, we update $\mu_{\max}=\mu$. This procedure is repeated until $|\mu_{\max}-\mu_{\min}|<\epsilon$ with $\epsilon=10^{-8}$. Since bisection method has a linear convergence rate [46], and in each iteration we need to compute K scalar functions $p_1^*(\mu),\cdots,p_K^*(\mu)$, the bisection method has a complexity of $\mathcal{O}(\log(1/\epsilon)K)$.

B. Scaling Law of Learning Centric Communication

According to **Proposition 2**, the user transmit power p_k is inversely proportional to the wireless channel gain $G_{k,k}=\|\mathbf{h}_k\|_2^2$. On the other hand, it is exponentially dependent on the classification error level μ and the learning parameters (a_k,b_k,D_k,A_k). Moreover, among all parameters, b_k is the most important factor, since b_k is involved in both the power and exponential functions. The above observations disclose that in edge machine learning systems, the learning parameters will have more significant impacts on the radio resource allocation than those of the wireless channels.

C. Learning Centric versus Communication Centric Power Allocation

Notice that the result in (16) is fundamentally different from the most well-known resource allocation schemes (e.g., iterative water-filling [15] and max-min fairness [16]). For example, the water-filling solution for maximizing the system throughput under $N\to+\infty$ is given by

$$p_k^{\mathrm{WF}}=\left(\frac{1}{\lambda\ln 2}-\frac{\sigma^2}{G_{k,k}}\right)^+,\tag{17}$$

where λ is a constant chosen such that $\sum_{k=1}^{K}p_k^{\mathrm{WF}}=P$. On the other hand, the max-min fairness solution under $N\to+\infty$ is given by

$$p_k^{\mathrm{FAIR}}=P\left(\sum_{k=1}^{K}\frac{\sigma^2}{G_{k,k}}\right)^{-1}\frac{\sigma^2}{G_{k,k}}.\tag{18}$$

It can be seen from (17) and (18) that the water-filling scheme would allocate more power resources to better channels, and the max-min fairness scheme would allocate more power resources to worse channels. But no matter which scheme we adopt, the only impact factor is the channel condition $\sigma^2/G_{k,k}$.

VI. Large-Scale Optimization Under IID Datasets

Although a KKT solution to P1 has been derived in Section IV, it can be seen that MM-based LCPA requires a cubic complexity with respect to K. This leads to time-consuming computations if K is in the range of hundreds or more. As a result, low-complexity large-scale optimization algorithms are indispensable. To this end, in this section we consider the case of $N\to+\infty$ under IID datasets, and develop an algorithm based on the FOM.

As $N\to+\infty$, we put $G_{k,l}=0$ for $k\neq l$ into Φ_m in P1, and the function Φ_m is asymptotically equal to

$$\Xi_m(\mathbf{p})=a_m\left[\sum_{k\in\mathcal{Y}_m}\frac{\xi BT}{D_m}\log_2\left(1+\frac{G_{k,k}p_k}{\sigma^2}\right)+A_m\right]^{-b_m}.\tag{19}$$

Therefore, the problem P1 in the case of IID datasets and $N\to+\infty$ is equivalent to

$$\begin{aligned}\mathrm{P3}:\ \min_{\mathbf{p}}\quad & \max_{m=1,\cdots,M}\ \beta_m\Xi_m(\mathbf{p}),\\ \text{s.t.}\quad & \sum_{k=1}^{K}p_k=P,\quad p_k\geq 0,\quad k=1,\cdots,K.\end{aligned}\tag{20}$$

The major challenge for solving P3 comes from the nonsmooth operator $\max$ in the objective function, which hinders us from computing the gradients. To deal with the non-smoothness, we reformulate P3 into a smooth bilevel optimization problem with ℓ_1-norm (simplex) constraints. Observing that the projection onto a simplex in Euclidean space requires high computational complexities, a mirror-prox LCPA method working on non-Euclidean manifold is proposed. In this way, the distance is measured by Kullback-Leibler (KL) divergence, and the non-Euclidean projection would have analytical expressions. Lastly, with an extragradient step, the proposed mirror-prox LCPA converges to the global optimal solution to P3 with an iteration complexity of $\mathcal{O}(1/\epsilon)$ [47]–[49], where ϵ is the target solution accuracy.

More specifically, we first equivalently transform P3 into a smooth bilevel optimization problem. By defining set

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

$\mathcal{P}=\left\{\mathbf{p}\in\mathbb{R}_+^{K\times 1}:\|\mathbf{p}\|_1=P\right\}$ and introducing variables $\boldsymbol{\alpha}\in\mathbb{R}^{M\times 1}$ such that $\boldsymbol{\alpha}\in\mathcal{A}=\{\boldsymbol{\alpha}\in\mathbb{R}_+^{M\times 1}:\|\boldsymbol{\alpha}\|_1=1\}$, P3 is rewritten as

$$\mathrm{P4}:\min_{\mathbf{p}\in\mathcal{P}}\ \max_{\boldsymbol{\alpha}\in\mathcal{A}}\ \underbrace{\sum_{m=1}^{M}\alpha_m\times\beta_m\Xi_m(\mathbf{p})}_{\Upsilon(\boldsymbol{\alpha},\mathbf{p})}. \tag{21}$$

It can be seen from P4 that $\Upsilon(\boldsymbol{\alpha},\mathbf{p})$ is differentiable with respect to either $\mathbf{p}$ or $\boldsymbol{\alpha}$, and the corresponding gradients are

$$\nabla_{\mathbf{p}}\Upsilon(\boldsymbol{\alpha},\mathbf{p})=\sum_{m=1}^{M}\alpha_m\beta_m\nabla_{\mathbf{p}}\Xi_m(\mathbf{p}), \tag{22a}$$

$$\nabla_{\boldsymbol{\alpha}}\Upsilon(\boldsymbol{\alpha},\mathbf{p})=\left[\beta_1\Xi_1(\mathbf{p}),\cdots,\beta_M\Xi_M(\mathbf{p})\right]^T, \tag{22b}$$

where

$$\nabla_{\mathbf{p}}\Xi_m(\mathbf{p})=\left[\frac{\partial\Xi_m}{\partial p_1},\cdots,\frac{\partial\Xi_m}{\partial p_K}\right]^T, \tag{23}$$

with its jth element being

$$\frac{\partial\Xi_m}{\partial p_j}=-\frac{a_m b_m\xi BT\,\mathbb{I}_{\mathcal{Y}_m}(j)}{D_m\ln 2\left(\sigma^2G_{j,j}^{-1}+p_j\right)}\times\left[\sum_{k\in\mathcal{Y}_m}\frac{\xi BT}{D_m}\log_2\left(1+\frac{G_{k,k}p_k}{\sigma^2}\right)+A_m\right]^{-b_m-1}. \tag{24}$$

However, P4 is a bilevel problem, with both the upper layer variable $\mathbf{p}$ and the lower layer variable $\boldsymbol{\alpha}$ involved in the simplex constraints. In order to facilitate the projection onto simplex constraints, below we consider a non-Euclidean (Banach) space induced by ℓ_1-norm. In such a space, the Bregman distance between two vectors $\mathbf{x}$ and $\mathbf{y}$ is the KL divergence

$$\mathbb{KL}\left(\mathbf{x},\mathbf{y}\right)=\sum_{l=1,2,\cdots}x_l\ln\left(\frac{x_l}{y_l}\right), \tag{25}$$

and the following proposition can be established.

Proposition 3: If the classification error $\mu=\max_m\beta_m\Xi_m(\mathbf{p})$ *is upper bounded by* $\mu\le\mu_0$*, then* $\Upsilon(\boldsymbol{\alpha},\mathbf{p})$ *is* $(L_1,L_2,L_2,0)$*–smooth in Banach space induced by* ℓ_1*-norm, where*

$$L_1=\max_{\substack{m=1,\cdots,M\\k=1,\cdots,K}}\frac{\beta_m a_m b_m\xi BTG_{k,k}}{D_m\ln 2\,\sigma^4}\left(\frac{\mu_0}{\beta_m a_m}\right)^{1+1/b_m}\times\left[G_{k,k}+\frac{(b_m+1)\xi BTH_m}{D_m\ln 2}\left(\frac{\mu_0}{\beta_m a_m}\right)^{1/b_m}\right], \tag{26a}$$

$$L_2=\max_{m=1,\cdots,M}\frac{\beta_m a_m b_m\xi BTH_m}{D_m\ln 2\,\sigma^2}\left(\frac{\mu_0}{\beta_m a_m}\right)^{1+1/b_m}, \tag{26b}$$

with $H_m:=\left\|\left[\mathbb{I}_{\mathcal{Y}_m}(1)\,G_{1,1},\cdots,\mathbb{I}_{\mathcal{Y}_m}(K)\,G_{K,K}\right]^T\right\|_2$.

Proof: See Appendix C. ∎

The smoothness result in **Proposition 3** enables us to apply mirror descent (i.e., generalized gradient descent in non-Euclidean space) to $\mathbf{p}$ and mirror ascent to $\boldsymbol{\alpha}$ in the ℓ_1-space [48]. This leads to the proposed mirror-prox LCPA, which is an iterative algorithm that involves i) a proximal step and ii) an extragradient step. In particular, the mirror-prox LCPA initially chooses a feasible $\mathbf{p}=\mathbf{p}^{[0]}\in\mathcal{P}$ and $\boldsymbol{\alpha}=\boldsymbol{\alpha}^{[0]}\in\mathcal{A}$ (e.g., $\mathbf{p}^{[0]}=P/K\,\mathbf{1}_K$ and $\boldsymbol{\alpha}^{[0]}=1/M\,\mathbf{1}_M$). Denoting the solution at the nth iteration as $(\mathbf{p}^{[n]},\boldsymbol{\alpha}^{[n]})$, the following equations are used to update the next-round solution [48]:

$$\mathbf{p}^{\diamond}=\operatorname*{argmin}_{\mathbf{p}\in\mathcal{P}}\ \mathbb{KL}\left(\mathbf{p},\mathbf{p}^{[n]}\right)+\eta\,\mathbf{p}^T\left[\sum_{m=1}^{M}\alpha_m^{[n]}\beta_m\nabla_{\mathbf{p}}\Xi_m(\mathbf{p}^{[n]})\right], \tag{27a}$$

$$\boldsymbol{\alpha}^{\diamond}=\operatorname*{argmin}_{\boldsymbol{\alpha}\in\mathcal{A}}\ \mathbb{KL}\left(\boldsymbol{\alpha},\boldsymbol{\alpha}^{[n]}\right)-\eta\left[\beta_1\Xi_1(\mathbf{p}^{[n]}),\cdots,\beta_M\Xi_M(\mathbf{p}^{[n]})\right]\boldsymbol{\alpha}, \tag{27b}$$

$$\mathbf{p}^{[n+1]}=\operatorname*{argmin}_{\mathbf{p}\in\mathcal{P}}\ \mathbb{KL}\left(\mathbf{p},\mathbf{p}^{[n]}\right)+\eta\,\mathbf{p}^T\left[\sum_{m=1}^{M}\alpha_m^{\diamond}\beta_m\nabla_{\mathbf{p}}\Xi_m(\mathbf{p}^{\diamond})\right], \tag{27c}$$

$$\boldsymbol{\alpha}^{[n+1]}=\operatorname*{argmin}_{\boldsymbol{\alpha}\in\mathcal{A}}\ \mathbb{KL}\left(\boldsymbol{\alpha},\boldsymbol{\alpha}^{[n]}\right)-\eta\left[\beta_1\Xi_1(\mathbf{p}^{\diamond}),\cdots,\beta_M\Xi_M(\mathbf{p}^{\diamond})\right]\boldsymbol{\alpha}, \tag{27d}$$

where η is the step-size, and the terms inside $[\cdots]$ in (27a)–(27d) are obtained from (22a)–(22b). Notice that a small η would lead to slow convergence of the algorithm while a large η would cause the algorithm to diverge. According to [48], η should be chosen inversely proportional to Lipschitz constant L_1 or L_2 derived in **Proposition 3**. In this paper, we set $\eta=10^3/L_2$ with $\mu_0=0.1$, which empirically provides fast convergence of the algorithm.

How Mirror-Prox LCPA Works: The formulas (27a)–(27b) update the variables along their gradient direction, while keeping the updated point $\{\mathbf{p}^{\diamond},\boldsymbol{\alpha}^{\diamond}\}$ close to the current point $\{\mathbf{p}^{[n]},\boldsymbol{\alpha}^{[n]}\}$. This is achieved via the *proximal operator* that minimizes the distance $\mathbb{KL}\left(\mathbf{p},\mathbf{p}^{[n]}\right)$ (or $\mathbb{KL}\left(\boldsymbol{\alpha},\boldsymbol{\alpha}^{[n]}\right)$) plus a first-order linear function. Since the KL divergence is the Bregman distance, the update (27a)–(27b) is a Bregman proximal step. On the other hand, the gradients in (27c) and (27d) are computed using $\mathbf{p}^{\diamond}$ and $\boldsymbol{\alpha}^{\diamond}$, respectively. By doing so, we can obtain the look-ahead gradient at the intermediate point $\mathbf{p}^{\diamond}$ and $\boldsymbol{\alpha}^{\diamond}$ for updating $\mathbf{p}^{[n+1]}$ and $\boldsymbol{\alpha}^{[n+1]}$. This "look-ahead" feature is called *extragradient*.

Lastly, we put the Bregman distance $\mathbb{KL}$ in (25), the function Ξ_m in (19), the gradient in (23), and a proper η into (27a)–(27d). Based on the KKT conditions, the equations (27a)–(27b) are shown to be equivalent to

$$p_k^{\diamond}=\left\{\sum_{i=1}^{K}p_i^{[n]}\exp\left[-\eta\left(\sum_{m=1}^{M}\alpha_m^{[n]}\beta_m\nabla_{p_i}\Xi_m(\mathbf{p}^{[n]})\right)\right]\right\}^{-1}\times P\,p_k^{[n]}\exp\left[-\eta\left(\sum_{m=1}^{M}\alpha_m^{[n]}\beta_m\nabla_{p_k}\Xi_m(\mathbf{p}^{[n]})\right)\right],\quad k=1,\cdots,K, \tag{28a}$$

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

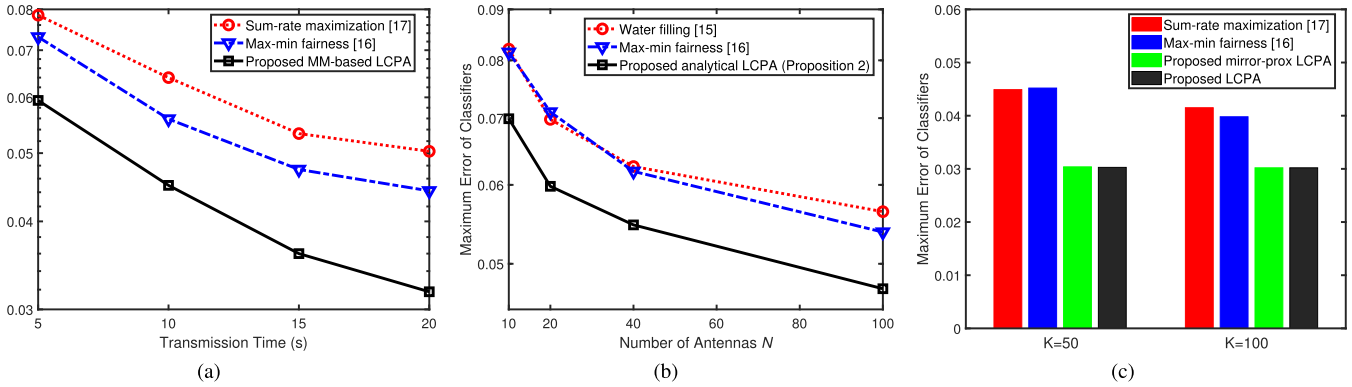


Fig. 5. a) Maximum error of classifiers versus total transmission time T when $N = 20$ and $K = 4$; b) Maximum error of classifiers versus the number of antennas N when $M = K = 2$; c) Maximum error of classifiers versus the number of users K when $N = 100$.

$$\alpha_m^{\diamond} = \left(\sum_{i=m}^{M} \alpha_i^{[n]} \exp\left[\eta\, \beta_i \Xi_i(\mathbf{p}^{[n]})\right]\right)^{-1} \times \alpha_m^{[n]} \exp\left[\eta\, \beta_m \Xi_m(\mathbf{p}^{[n]})\right], \quad m = 1, \cdots, M. \tag{28b}$$

The equations (27c)–(27d) can be similarly reduced to closed-form expressions.

According to **Proposition 3** and [47], the mirror-prox LCPA algorithm is guaranteed to converge to the optimal solution to P4. But in practice, we can terminate the iterative procedure when the norm $\left\|\mathbf{p}^{[n]} - \mathbf{p}^{[n-1]}\right\|_\infty$ is small enough, e.g., $\left\|\mathbf{p}^{[n]} - \mathbf{p}^{[n-1]}\right\|_\infty < 10^{-8}$. The entire procedure for computing the solution to P4 using the mirror-prox LCPA is summarized in the right hand branch of Fig. 4c. In terms of computational complexity, computing (28a) requires a complexity of $\mathcal{O}(MK)$. Since the number of iterations for the mirror-prox LCPA to converge is $\mathcal{O}(1/\epsilon)$ with ϵ being the target solution accuracy, the total complexity of mirror-prox LCPA would be $\mathcal{O}(MK/\epsilon)$.

VII. Simulation Results and Discussions

This section provides simulation results to evaluate the performance of the proposed algorithms. It is assumed that the noise power $\sigma^2 = -87$ dBm (corresponding to power spectral density -140 dBm/Hz with 180 kHz bandwidth [27]), which includes thermal noise and receiver noise. The total transmit power at users is set to $P = 13$ dBm (i.e., 20 mW), with the communication bandwidth $B = 180$ kHz. The path loss of the kth user $\varrho_k = -100$ dB is adopted [40], and $\mathbf{h}_k$ is generated according to $\mathcal{CN}(\mathbf{0}, \varrho_k \mathbf{I}_N)$ [44]. Without otherwise specified, it is assumed that $\xi = 1$. We set $Z_k^{\min} = 0$ and $Z_k^{\max} = +\infty$ for all k in Sections VII-A to VII-C. Simulations with upper and lower bounds on data amount will be presented in Section VII-D. Each point in the figures is obtained by averaging over 10 simulation runs, with independent channels in each run. All optimization problems are solved by Matlab R2015b on a desktop with Intel Core i5-4570 CPU at 3.2 GHz and 8 GB RAM. All the classifiers are trained by Python 3.6 on a GPU server with Intel Core i7-6800 CPU at 3.4 GHz and GeForce GTX 1080 GPU.

A. CNN and SVM

We consider the aforementioned CNN and SVM classifiers with the number of learning models $M = 2$ at the edge: i) Classification of MNIST dataset [30] via CNN; ii) Classification of digits dataset in Scikit-learn [34] via SVM. The data size of each sample is $D_1 = 6276$ bits for the MNIST dataset and $D_2 = 324$ bits for the digits dataset in Scikit-learn. It is assumed that there are $A_1 = 300$ CNN samples and $A_2 = 200$ SVM samples in the historical dataset. The parameters in the two classification error models are assumed to be perfectly known and they are given by $(a_1, b_1) = (7.3, 0.69)$ for CNN and $(a_2, b_2) = (5.2, 0.72)$ for SVM as in Fig. 3b. Finally, it is assumed that $(\beta_1, \beta_2) = (1, 1.2)$ since the approximation error of SVM in Fig. 3b is larger than that of CNN.

To begin with, the case of $N = 20$ and $K = 4$ with $\mathcal{Y}_1 = \{1\}$ and $\mathcal{Y}_2 = \{2, 3, 4\}$ is simulated. Under the above settings, we compute the collected sample sizes by executing the proposed MM-based LCPA, and the maximum error of classifiers (i.e., the worse classification result of CNN and SVM, where the classification error for each task is obtained from the machine learning experiment using the sample sizes from the power allocation algorithms) versus the total transmission time T is shown in Fig. 5a. Besides the proposed MM-based LCPA, we also simulate two benchmark schemes: 1) Max-min fairness scheme [16, Sec. II-C]; 2) Sum-rate maximization scheme [17, Sec. IV]. It can be seen from Fig. 5a that the proposed MM-based LCPA algorithm with $\mathcal{I} = 10$ has a significantly smaller classification error compared to other schemes, and the gap concisely quantifies the benefit brought by more training images for CNN under joint communication and learning design. For example, at $T = 20$ in Fig. 5a, the proposed MM-based LCPA collects 2817 MNIST images on average, while the sum-rate maximization and the max-min fairness schemes collect 1686 images and 1781 images, respectively. Furthermore, if we target at the same learning error, the proposed algorithm saves the transmission time by at least 30% compared to benchmark schemes. This can be seen from Fig. 5a at the target error 4.5%, where the proposed algorithm takes 10 seconds for transmission, but other methods require about 20 seconds. The saved time enables the edge to collect data for other edge computing tasks [50].

To get more insight into the edge learning system, the case of $K = 2$ with $\mathcal{Y}_1 = \{1\}$ and $\mathcal{Y}_2 = \{2\}$ at $T = 5$ s is simulated, and the classification error versus the number of antennas $N = \{10, 20, 40, 100\}$ is shown in Fig. 5b. It can be seen from Fig. 5b that the classification error decreases

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

as the number of antennas increases, which demonstrates the advantage of employing massive MIMO in edge machine learning. More importantly, the proposed analytical solution in **Proposition 2** outperforms the water-filling[3] and max-min fairness schemes even at a relatively small number of antennas $N = 10$. This is achieved by allocating much more power resources to the first user (i.e., user uploading datasets for CNN), because training CNN is more difficult than training SVM. In particular, the transmit powers in mW are given by: 1) $(p_1, p_2) = (19.8473, 0.1524)$ for the analytical LCPA scheme; 2) $(p_1, p_2) = (9.9862, 10.0138)$ for the water-filling scheme; and 3) $(p_1, p_2) = (10.0869, 9.9131)$ for the max-min fairness scheme. Notice that the performance gain brought by LCPA in Fig. 5b is slightly smaller than that in Fig. 5a, since the ratio $|\mathcal{Y}_1|/|\mathcal{Y}_2|$ is increased. But no matter what value $|\mathcal{Y}_1|$ and $|\mathcal{Y}_2|$ take, the proposed LCPA always outperforms existing algorithms due to its learning centric feature.

To verify the performance and the low complexity nature of the mirror-prox LCPA in Section VI when the number of antennas is large, the case of $N = 100$ and $K \in \{50, 100\}$ at $T = 5$ s is simulated, with $\mathcal{Y}_1$ containing the first $1/5$ users and $\mathcal{Y}_2$ containing the rest $4/5$ users. The maximum error of classifiers versus the number of users K is shown in Fig. 5c. It can be seen that the proposed mirror-prox LCPA algorithm significantly reduces the classification error compared to the water-filling and max-min fairness schemes, and it achieves performance close to that of the MM-based LCPA. On the other hand, the average execution time at $K = 100$ is given by: 1) 18.2496 s for the MM-based LCPA; 2) 0.5673 s for the mirror-prox LCPA; 3) 0.0044 s for the water-filling scheme; and 4) 0.0054 s for the max-min fairness scheme. Compared with MM-based LCPA, the mirror-prox LCPA saves at least 95% of the computation time, which corroborates the linear complexity derived in Section VI.

B. Deep Neural Networks

Next, we consider the ResNet-110 as task 1 and the CNN in Section III as task 2 at the edge, with $D_1 = 24584$ bits and $D_2 = 6276$ bits. The error rate parameters are given by $(a_1, b_1) = (8.15, 0.44)$ and $(a_2, b_2) = (7.3, 0.69)$. In addition, it is assumed that $(\beta_1, \beta_2) = (1, 1)$, $T = 200$ s, and there is no historical sample at the edge (i.e., $A_1 = A_2 = 0$). We simulate the case of $N = 20$ and $K = 4$ with $\mathcal{Y}_1 = \{1, 2\}$ and $\mathcal{Y}_2 = \{3, 4\}$. For ResNet-110, we assume that the datasets $\{\mathcal{D}_1, \mathcal{D}_2\}$ are formed by dividing the CIFAR-10 dataset into two parts, each with 30000 different samples. For CNN, we assume that the datasets $\{\mathcal{D}_3, \mathcal{D}_4\}$ are formed by dividing the MNIST dataset into two parts, each also with 30000 different samples. The worse classification error between the two tasks (obtained from the machine learning experiment using the sample sizes from the power allocation algorithms) is: 1) 14.13% for MM-based LCPA; 2) 16.79% for the sum-rate maximization scheme; and 3) 16.42% for the max-min fairness scheme. It can be seen that the proposed LCPA achieves the smallest classification error, which demonstrates the effectiveness of learning centric resource allocation under deep neural networks and large datasets.

[3]In the case of large N, sum-rate maximization scheme [17, Sec. IV] would reduce to the iterative water-filling scheme [15], which allocates power according to (17).

C. Practical Considerations

In Sections VII-A and VII-B, we have assumed that the parameters (a_m, b_m) are perfectly known. In practice, we need to estimate them from the historical data. More specifically, denote the historical data as $\mathcal{H}_k \subseteq \mathcal{D}_k$. We take $2/3$ of the historical data in $\mathcal{H}_k$ as the training dataset $\mathcal{T}_k$, and the remaining $1/3$ as the validation dataset $\mathcal{V}_k$ for all k. Therefore, the number of training samples for estimating (a_m, b_m) is $\sum_{k\in\mathcal{Y}_m} |\mathcal{T}_k|$. Based on the above dataset partitioning, the parameters $\{a_m, b_m\}$ are estimated in two stages:

- In the training stage, we can vary the sample size v_m as $(v_m^{(1)}, v_m^{(2)}, \cdots)$ within $[0, \sum_{k\in\mathcal{Y}_m} |\mathcal{T}_k|]$. For each $v_m^{(i)}$, we train the learning model for E epochs and test the trained model on the validation dataset $\{\mathcal{V}_k\}_{k\in\mathcal{Y}_m}$. The classification errors corresponding to the different sample sizes are given by $(\Psi_m(v_m^{(1)}), \Psi_m(v_m^{(2)}), \cdots)$.
- In the fitting stage, with the classification error versus sample size, the parameters (a_m, b_m) in Θ_m are found via the nonlinear least squares fitting in (9).

Assuming that the complexity of processing each image (e.g., for CNN, each processing stage includes a backward pass and a forward pass) is $\mathcal{O}(W_m)$ where W_m is the number of parameters in the learning model, the complexity in the training stage is $\mathcal{O}(EW_m \sum_i v_m^{(i)})$. If $\sum_i v_m^{(i)}$ is smaller than the total number of samples after data collection, then $\mathcal{O}(EW_m \sum_i v_m^{(i)})$ would be smaller than the actual model training complexity. On the other hand, the complexity in the fitting stage is negligible, since the problem (9) only has two variables. Even using a naive brute-force search with a step size of 0.001 for each dimension, the running time for solving (9) is only 5 seconds on a desktop with Intel Core i5-4570 CPU at 3.2 GHz and 8 GB RAM.

To illustrate the above procedure, we consider the case of $N = 100$ and $K = 5$ with $M = 2$, where the first task is to train the CNN with user $\mathcal{Y}_1 = \{1\}$ and the second task is to train the SVM with users $\mathcal{Y}_1 = \{2, 3, 4, 5\}$. For CNN, we assume that the dataset $\mathcal{D}_1$ contains 10000 different samples from the MINST dataset. For SVM, we assume that the datasets $\{\mathcal{D}_2, \mathcal{D}_3, \mathcal{D}_4, \mathcal{D}_5\}$ are formed by dividing the scikit-learn digit dataset into four parts, each with 250 different samples. To conform with the size of $\mathcal{D}_1$, the datasets $\{\mathcal{D}_2, \mathcal{D}_3, \mathcal{D}_4, \mathcal{D}_5\}$ are augmented by appending 9750 samples (generated via random rotations of the original samples) to the 250 original samples. The test dataset for CNN contains another 1000 samples from the MNIST dataset and the test dataset for SVM contains another 797 samples from the scikit-learn digits dataset. It is assumed that $|\mathcal{H}_1| = 450$ and $|\mathcal{H}_2| = \cdots = |\mathcal{H}_5| = 75$. According to the "$2/3$-training and $1/3$-validation" partitioning of $\mathcal{H}_k$, we have $|\mathcal{T}_1| = 300$ and $|\mathcal{T}_2| = \cdots = |\mathcal{T}_5| = 50$, meaning that there are 300 and 200 historical samples for training CNN and SVM, respectively. For CNN, we vary the sample size v_1 as $(v_1^{(1)}, v_1^{(2)}, \cdots) = (100, 150, 200, 300)$ and perform

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

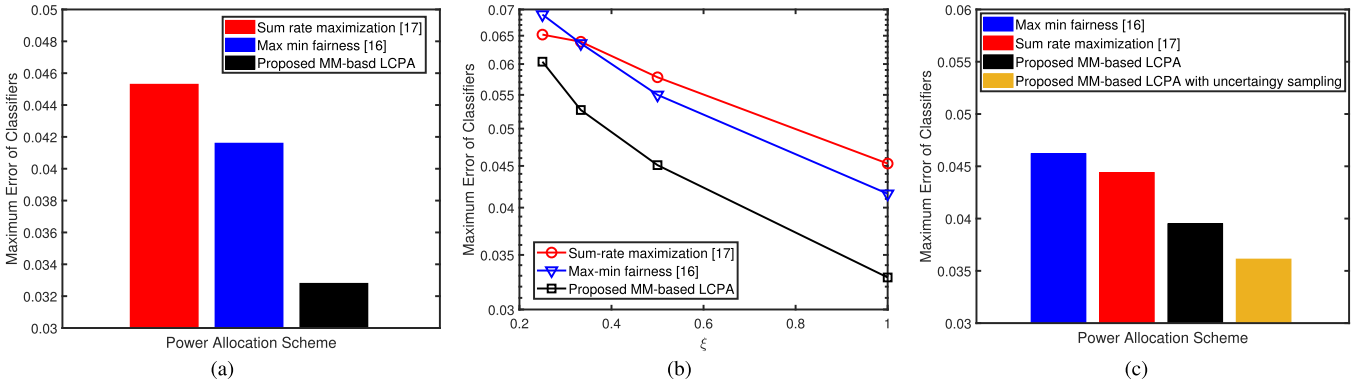


Fig. 6. a) Comparison of classification error when $N = 100$ and $K = 5$ at $T = 10$ s with (a_m, b_m) estimated from historical data; b) Maximum error of classifiers versus ξ when $N = 100$ and $K = 5$; c) Comparison of classification error when $N = 100$ and $K = 6$ at $T = 10$ s in the non-IID case.

training for $E = 200$ epochs. The classification errors on the validation dataset $\mathcal{V}_1$ are given by $(0.300, 0.200, 0.140, 0.120)$, and after model fitting we have $(a_1, b_1) = (9.74, 0.77)$. On the other hand, for SVM, we vary the sample size v_2 as $(v_2^{(1)}, v_2^{(2)}, \cdots) = (30, 50, 100, 200)$. The classification errors on the validation dataset $\{\mathcal{V}_2, \mathcal{V}_3, \mathcal{V}_4, \mathcal{V}_5\}$ are given by $(0.510, 0.280, 0.220, 0.050)$, and after model fitting we have $(a_2, b_2) = (14.27, 0.85)$.

Based on the above estimation results for (a_m, b_m), the classification errors at $T = 10$ s for different schemes are compared in Fig. 6a. It can be seen from Fig. 6a that the proposed LCPA reduces the minimum classification error by at least 20% compared with other simulated schemes. To demonstrate the performance of the proposed LCPA under various values of ξ, the classification error versus $\xi = \{1/4, 1/3, 1/2, 1\}$ at $T = 10$ s is shown in Fig. 6b. It can be seen from Fig. 6b that the classification error increases as ξ decreases, which is due to the loss of samples during wireless transmission. However, no matter what value ξ takes, the proposed LCPA achieves the minimum classification error among all the simulated schemes.

D. Non-IID Dataset

In the non-IID case, we add one more user (denoted as user 6) to the system considered in Section VII-C, with the first task training the CNN with users $\mathcal{Y}_1 = \{1, 6\}$ and the second task training the SVM with users $\mathcal{Y}_1 = \{2, 3, 4, 5\}$. The dataset $\mathcal{D}_6$ repeats the first sample in the MINST dataset for 10000 times and $|\mathcal{H}_6| = 450$. Out of this $\mathcal{H}_6$, we use $2/3$ as the training dataset $\mathcal{T}_6$ and $1/3$ as the validation dataset $\mathcal{V}_6$. We train the CNN in Fig. 1 using the training dataset $\{\mathcal{T}_1, \mathcal{T}_6\}$ for 200 epochs, and obtain the parameter $\mathbf{w}$ in the CNN. We then compute the prediction confidence for all the data $\mathbf{d} \in \{\mathcal{V}_1, \mathcal{V}_6\}$ based on equation (6) with the probability function being the softmax output of CNN. It turns out that the least confident samples in $\mathcal{V}_1$ and $\mathcal{V}_6$ have the following scores: $\min_{\mathbf{d}\in\mathcal{V}_1} U(\mathbf{d}) = 0.274$ and $\min_{\mathbf{d}\in\mathcal{V}_6} U(\mathbf{d}) = 0.999$. For user 1, since the CNN model is not sure about its prediction, we set $Z_1^{\min} = 100$ and $Z_1^{\max} = 10000$. For user 6, since the CNN model can predict its validation data with very high confidence, we set $Z_6^{\min} = 0$ and $Z_6^{\max} = 10$. Based on the ratio between $Z_1^{\max}$ and $Z_6^{\max}$, we use 450 samples from $\mathcal{H}_1$ and $10/10000 \times 450$ samples from $\mathcal{H}_6$ for estimating (a_m, b_m). Then we have $(a_1, b_1) = (9.74, 0.77)$. For the SVM model, since the data from users $\{2, 3, 4, 5\}$ are assumed to be IID, (a_2, b_2) is the same as that in Section VII-C, i.e., $(a_2, b_2) = (14.27, 0.85)$.

The comparison of classification error in the non-IID case is shown in Fig. 6c. It can be seen from Fig. 6c that the performance of MM-based LCPA (without uncertainty sampling) becomes worse than that in Fig. 6a, which is due to the additional resources allocated to user 6. However, the proposed MM-based LCPA still outperforms the sum-rate maximization and max-min fairness schemes. More importantly, by adopting the uncertainty sampling method to distinguish the users' data quality, the classification error of LCPA can be further reduced (e.g., from the black bar to the yellow bar in Fig. 6c). This demonstrates the benefit brought by *joint estimation of sample size and prediction confidence* in edge machine learning systems.

VIII. Conclusion

This paper has introduced the LCPA concept to edge machine learning. By adopting an empirical classification error model, learning efficient edge resource allocation has been obtained via the MM-based LCPA algorithm. In the large-scale settings, a fast FOM has been derived to tackle the curse of high-dimensionality. Simulation results have shown that the proposed LCPA algorithms achieve lower classification errors than existing power allocation schemes. Furthermore, the proposed fast algorithm significantly reduces the execution time compared with the MM-based LCPA while still achieving satisfactory performance.

Appendix A
Proof of Proposition 1

To prove part (i), consider the following inequality:

$$-\ln\left(\sum_{l=1,l\neq k}^{K}\frac{G_{k,l}p_l^\star}{\sigma^2}+1\right)-\frac{\sum_{l=1,l\neq k}^{K}G_{k,l}p_l/\sigma^2+1}{\sum_{l=1,l\neq k}^{K}G_{k,l}p_l^\star/\sigma^2+1}+1 \leq -\ln\left(\sum_{l=1,l\neq k}^{K}\frac{G_{k,l}p_l}{\sigma^2}+1\right), \quad (29)$$

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

which is obtained from $-\ln(x') - \frac{1}{x'}(x - x') \leq -\ln(x)$ for any (x, x') due to the convexity of $-\ln(x)$. Adding $\ln\left(\sum_{l=1}^{K} G_{k,l}p_l/\sigma^2 + 1\right)$ on both sides of (29), we obtain

$$\begin{aligned}
&\ln\left(\sum_{l=1}^{K}\frac{G_{k,l}p_l}{\sigma^2}+1\right)-\ln\left(\sum_{l=1,l\neq k}^{K}\frac{G_{k,l}p_l^\star}{\sigma^2}+1\right)\\
&\quad-\frac{\sum_{l=1,l\neq k}^{K}G_{k,l}p_l/\sigma^2+1}{\sum_{l=1,l\neq k}^{K}G_{k,l}p_l^\star/\sigma^2+1}+1\\
&\leq \ln\left(\sum_{l=1}^{K}\frac{G_{k,l}p_l}{\sigma^2}+1\right)-\ln\left(\sum_{l=1,l\neq k}^{K}\frac{G_{k,l}p_l}{\sigma^2}+1\right)\\
&=\ln\left(1+\frac{G_{k,k}p_k}{\sum_{l=1,l\neq k}^{K}G_{k,l}p_l+\sigma^2}\right). \qquad (30)
\end{aligned}$$

Putting the result of (30) into $\widetilde{\Phi}_m(\mathbf{p}|\mathbf{p}^\star)$ in (12) and since $a_m x^{-b_m}$ is a decreasing function of x, we immediately prove

$$\begin{aligned}
&\widetilde{\Phi}_m(\mathbf{p}|\mathbf{p}^\star)\\
&\geq a_m\left[\sum_{k\in\mathcal{Y}_m}\frac{\xi BT}{D_m\ln 2}\ln\left(1+\frac{G_{k,k}p_k}{\sum_{l\neq k}G_{k,l}p_l+\sigma^2}\right)+A_m\right]^{-b_m}\\
&=\Phi_m(\mathbf{p}). \qquad (31)
\end{aligned}$$

To prove part (ii), we first notice that $\widetilde{\Phi}_m(\mathbf{p}|\mathbf{p}^\star) = h_m\left(g_m(\mathbf{p}|\mathbf{p}^\star)\right)$ is a composition function of $h_m \circ g_m$, where $h_m(x) = a_m x^{-b_m}$ and

$$\begin{aligned}
g_m(\mathbf{p}|\mathbf{p}^\star)=\sum_{k\in\mathcal{Y}_m}\frac{\xi BT}{D_m\ln 2}\Bigg[&\ln\left(\sum_{l=1}^{K}\frac{G_{k,l}p_l}{\sigma^2}+1\right)\\
&-\ln\left(\sum_{l=1,l\neq k}^{K}\frac{G_{k,l}p_l^\star}{\sigma^2}+1\right)\\
&-\frac{\sum_{l=1,l\neq k}^{K}G_{k,l}p_l/\sigma^2+1}{\sum_{l=1,l\neq k}^{K}G_{k,l}p_l^\star/\sigma^2+1}+1\Bigg]+A_m. \qquad (32)
\end{aligned}$$

Since $\nabla h_m(x) = -a_m b_m x^{-b_m-1} \leq 0$ and $\nabla^2\ h_m(x) = a_m b_m(b_m+1)x^{-b_m-2} \geq 0$, the function $h_m(x)$ is convex and nonincreasing. Adding to the fact that $g_m(\mathbf{p}|\mathbf{p}^\star)$ is a concave function of $\mathbf{p}$, we immediately prove the convexity of $\widetilde{\Phi}_m$ using the composition rule [41, Ch. 3, pp. 84].

Finally, to prove $\widetilde{\Phi}_m(\mathbf{p}^\star|\mathbf{p}^\star) = \Phi_m(\mathbf{p}^\star)$ and $\nabla_{\mathbf{p}}\widetilde{\Phi}_m(\mathbf{p}^\star|\mathbf{p}^\star) = \nabla_{\mathbf{p}}\Phi_m(\mathbf{p}^\star)$, we put $\mathbf{p} = \mathbf{p}^\star$ into the functions $\{\widetilde{\Phi}_m, \Phi_m, \nabla_{\mathbf{p}}\widetilde{\Phi}_m, \nabla_{\mathbf{p}}\Phi_m\}$, and we immediately obtain part (iii).

APPENDIX B
PROOF OF PROPOSITION 2

To prove this proposition, the Lagrangian of P2 is

$$\begin{aligned}
L=\mu+\sum_{k=1}^{K}\nu_k\Bigg[\beta_k a_k\left(\frac{\xi BT}{D_k}\log_2\left(1+\frac{G_{k,k}p_k}{\sigma^2}\right)+A_k\right)^{-b_k}\\
-\mu\Bigg]+\chi\left(\sum_{k=1}^{K}p_k-P\right)-\sum_{k=1}^{K}\theta_k p_k, \qquad (33)
\end{aligned}$$

where $\{\nu_k, \chi, \theta_k\}$ are non-negative Lagrange multipliers. According to the KKT conditions $\partial L/\partial\mu^* = 0$ and $\partial L/\partial p_k^* = 0$ [41], the optimal $\{\mu^*, p_k^*, \nu_k^*, \chi^*, \theta_k^*\}$ must together satisfy

$$1-\sum_{k=1}^{K}\nu_k^*=0, \quad \chi^*+\nu_k^*\times F_k(p_k^*)=\theta_k^*, \quad \forall k, \qquad (34)$$

where

$$\begin{aligned}
F_k(x)=&-\frac{\beta_k a_k b_k \xi BT}{D_k\ln 2\left(\sigma^2 G_{k,k}^{-1}+x\right)}\\
&\times\left[\frac{\xi BT}{D_k}\log_2\left(1+\frac{G_{k,k}x}{\sigma^2}\right)+A_k\right]^{-b_k-1}, \qquad (35)
\end{aligned}$$

with $x \geq 0$ ($x \neq 0$ if $A_k = 0$). Notice that $F_k(x) < 0$ holds for any $x \geq 0$. Based on the result of (34), it is clear that $\sum_{k=1}^{K}\frac{\theta_k^*-\chi^*}{F_k(p_k^*)} = 1$. Adding to the fact that $\theta_k^* \geq 0$ and $F_k(p_k^*) < 0$, we must have $\chi^* \neq 0$. Now we will consider two cases.

- $p_k^* = 0$. In this case, $\beta_k a_k u_k^{-b_k} \leq \mu^*$ must hold.
- $p_k^* > 0$. In such a case, based on the complementary slackness condition, we must have $\theta_k^* = 0$. Putting $\theta_k^* = 0$ into (34) and using $\chi^* \neq 0$, $\nu_k^* \neq 0$ holds. Using $\nu_k^* \neq 0$ and the complementary slackness condition, $\beta_k a_k\left(\frac{\xi BT}{D_k}\log_2\left(1+\frac{G_{k,k}p_k^*}{\sigma^2}\right)+A_k\right)^{-b_k} = \mu^*$ for all k.

Combining the above two cases gives (16) and the proposition is proved.

APPENDIX C
PROOF OF PROPOSITION 3

To prove this proposition, we need the following lemma for $\nabla_{\mathbf{p}}\Xi_m(\mathbf{p})$.

Lemma 1: If $\mu \leq \mu_0$, the gradient function $\nabla_{\mathbf{p}}\Xi_m(\mathbf{p})$ satisfies the following:

(i) $\|\nabla_{\mathbf{p}}\Xi_m(\mathbf{p})\|_2 \leq L_2/\beta_m$;

(ii) $\|\nabla_{\mathbf{p}}\Xi_m(\mathbf{p}) - \nabla_{\mathbf{p}}\Xi_m(\mathbf{p}')\|_\infty \leq L_1/\beta_m\|\mathbf{p}-\mathbf{p}'\|_2$.

Proof: To begin with, the assumption $\mu = \max_m \beta_m\Xi_m(\mathbf{p}) \leq \mu_0$ gives

$$\sum_{k\in\mathcal{Y}_m}\frac{\xi BT}{D_m}\log_2\left(1+\frac{G_{k,k}p_k}{\sigma^2}\right)+A_m\geq\left(\frac{\beta_m a_m}{\mu_0}\right)^{\frac{1}{b_m}}. \qquad (36)$$

Based on (36) and the expression of $\partial\Xi_m/\partial p_j$ in (24), we have

$$\begin{aligned}
\left|\frac{\partial\Xi_m}{\partial p_j}\right| &\leq a_m b_m\left(\frac{\beta_m a_m}{\mu_0}\right)^{-1-1/b_m}\frac{\xi BT\mathbb{I}_{\mathcal{Y}_m}(j)}{D_m\ln 2\left(\sigma^2 G_{j,j}^{-1}+p_j\right)}\\
&\leq\frac{a_m b_m\xi BT G_{j,j}\mathbb{I}_{\mathcal{Y}_m}(j)}{D_m\ln 2\,\sigma^2}\left(\frac{\mu_0}{\beta_m a_m}\right)^{1+1/b_m}, \qquad (37)
\end{aligned}$$

where the second inequality is due to $p_j \geq 0$. Putting the above result into (23), and based on the definition of L_2 in (26b), part (i) is immediately proved.

Next, to prove part (ii), we notice that the derivative in (24) can be rewritten as $\nabla_{p_j}\Xi_m(\mathbf{p}) = h_m(\mathbf{p})\times g_{m,j}(\mathbf{p})$, with the auxiliary functions

$$h_m(\mathbf{p})=-a_m b_m\left[\sum_{k\in\mathcal{Y}_m}\frac{\xi BT}{D_m}\log_2\left(1+\frac{G_{k,k}p_k}{\sigma^2}\right)+A_m\right]^{-b_m-1}, \qquad (38a)$$

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

$$g_{m,j}(\mathbf{p}) = \frac{\xi BT\mathbb{I}_{\mathcal{Y}_m}(j)}{D_m \ln 2\ (\sigma^2 G_{j,j}^{-1} + p_j)}. \tag{38b}$$

Using the result in (36) and due to $p_j \geq 0$, we have

$$\begin{aligned} |h_m(\mathbf{p})| &\leq a_m b_m \left(\frac{\mu_0}{\beta_m a_m}\right)^{1+1/b_m}, \\ |g_{m,j}(\mathbf{p})| &\leq \frac{\xi BTG_{j,j}\mathbb{I}_{\mathcal{Y}_m}(j)}{D_m \ln 2\, \sigma^2}. \end{aligned} \tag{39}$$

Furthermore, according to Lipschitz conditions [48] of h_m and $g_{m,j}$, they satisfy

$$\begin{aligned} |h_m(\mathbf{p}) - h_m(\mathbf{p}')| &\leq \sup_{\mathbf{p}\in\mathcal{P}} \|\nabla_{\mathbf{p}} h_m(\mathbf{p})||_2 \times ||\mathbf{p} - \mathbf{p}'\|_2 \\ &\leq \frac{a_m b_m (b_m+1)\xi BTH_m}{D_m \ln 2\, \sigma^2} \\ &\quad \times \left(\frac{\mu_0}{\beta_m a_m}\right)^{1+2/b_m} \|\mathbf{p} - \mathbf{p}'\|_2, \end{aligned} \tag{40a}$$

$$\begin{aligned} |g_{m,j}(\mathbf{p}) - g_{m,j}(\mathbf{p}')| &\leq \sup_{\mathbf{p}\in\mathcal{P}} \|\nabla_{\mathbf{p}} g_{m,j}(\mathbf{p})\|_2 \times \|\mathbf{p} - \mathbf{p}'\|_2 \\ &\leq \frac{\xi BTG_{j,j}^2 \mathbb{I}_{\mathcal{Y}_m}(j)}{D_m \ln 2\, \sigma^4} \|\mathbf{p} - \mathbf{p}'\|_2. \end{aligned} \tag{40b}$$

As a result, the following inequality is obtained:

$$\begin{aligned} &|\nabla_{p_j}\Xi_m(\mathbf{p}) - \nabla_{p_j}\Xi_m(\mathbf{p}')| \\ &\leq |h_m(\mathbf{p})|\,|g_{m,j}(\mathbf{p}) - g_{m,j}(\mathbf{p}')| \\ &\quad + |h_m(\mathbf{p}) - h_m(\mathbf{p}')|\,|g_{m,j}(\mathbf{p}')| \\ &\leq \left[a_m b_m \left(\frac{\mu_0}{\beta_m a_m}\right)^{1+1/b_m} \frac{\xi BTG_{j,j}^2}{D_m \ln 2\, \sigma^4} + \left(\frac{\mu_0}{\beta_m a_m}\right)^{1+2/b_m} \right. \\ &\quad \left. \times \frac{a_m b_m (b_m+1)\xi^2 B^2 T^2 G_{j,j} H_m}{D_m^2 \ln^2 2\sigma^4} \right] \|\mathbf{p} - \mathbf{p}'\|_2 \\ &\leq L_1/\beta_m \|\mathbf{p} - \mathbf{p}'\|_2, \end{aligned} \tag{41}$$

where the first inequality is due to $|ab+cd| \leq |a||b|+|c||d|$, and the second inequality is obtained from (39) and (40a)–(40b). By taking the maximum of (41) for all j, part (ii) is proved. ∎

Based on **Lemma 1**, we are now ready to prove the proposition. In particular, according to [47]–[49], the function $\Upsilon(\boldsymbol{\alpha}, \mathbf{p})$ is $(L_1, L_2, L_2, 0)$–smooth if and only if

$$\begin{aligned} &\left\| \left[\sum_{m=1}^{M} \alpha_m\beta_m \nabla_{\mathbf{p}}\Xi_m(\mathbf{p})\right] - \left[\sum_{m=1}^{M} \alpha_m\beta_m \nabla_{\mathbf{p}}\Xi_m(\mathbf{p}')\right] \right\|_\infty \\ &\leq L_1 \|\mathbf{p} - \mathbf{p}'\|_1, \end{aligned} \tag{42a}$$

$$\begin{aligned} &\left\| \left[\sum_{m=1}^{M} \alpha_m\beta_m \nabla_{\mathbf{p}}\Xi_m(\mathbf{p})\right] - \left[\sum_{m=1}^{M} \alpha'_m\beta_m \nabla_{\mathbf{p}}\Xi_m(\mathbf{p})\right] \right\|_\infty \\ &\leq L_2 \|\boldsymbol{\alpha} - \boldsymbol{\alpha}'\|_1, \end{aligned} \tag{42b}$$

$$\begin{aligned} &\left\| [\beta_1\Xi_1(\mathbf{p}), \cdots, \beta_M\Xi_M(\mathbf{p})] \right. \\ &\left. - [\beta_1\Xi_1(\mathbf{p}'), \cdots, \beta_M\Xi_M(\mathbf{p}')] \right\|_\infty \leq L_2 \|\mathbf{p} - \mathbf{p}'\|_1, \end{aligned} \tag{42c}$$

$$\begin{aligned} &\left\| [\beta_1\Xi_1(\mathbf{p}), \cdots, \beta_M\Xi_M(\mathbf{p})] \right. \\ &\left. - [\beta_1\Xi_1(\mathbf{p}), \cdots, \beta_M\Xi_M(\mathbf{p})] \right\|_\infty \leq 0\ \|\boldsymbol{\alpha} - \boldsymbol{\alpha}'\|_1, \end{aligned} \tag{42d}$$

for any $\mathbf{p}, \mathbf{p}' \in \mathcal{P}$ and $\boldsymbol{\alpha}, \boldsymbol{\alpha}' \in \mathcal{A}$. To prove the above inequalities, we need to upper bound the left hand sides of (42a)–(42d) and compare the bounds with the right hand sides of (42a)–(42d). To this end, the left hand side of (42a) is bounded as

$$\begin{aligned} &\left\| \sum_{m=1}^{M} \alpha_m\beta_m \left[\nabla_{\mathbf{p}}\Xi_m(\mathbf{p}) - \nabla_{\mathbf{p}}\Xi_m(\mathbf{p}')\right] \right\|_\infty \\ &\leq \sum_{m=1}^{M} \alpha_m\beta_m \|\nabla_{\mathbf{p}}\Xi_m(\mathbf{p}) - \nabla_{\mathbf{p}}\Xi_m(\mathbf{p}')\|_\infty \\ &\leq L_1 \|\mathbf{p} - \mathbf{p}'\|_2, \end{aligned} \tag{43}$$

where the last inequality is from **Lemma 1**. Further due to $\|\mathbf{p} - \mathbf{p}'\|_2 \leq \|\mathbf{p} - \mathbf{p}'\|_1$, the equation (42a) is proved. On the other hand, the left hand side of (42b) is upper bounded as

$$\begin{aligned} &\left\| \sum_{m=1}^{M} (\alpha_m - \alpha'_m)\beta_m \nabla_{\mathbf{p}}\Xi_m(\mathbf{p}) \right\|_\infty \\ &\leq \sum_{m=1}^{M} |\alpha_m - \alpha'_m|\, \|\beta_m \nabla_{\mathbf{p}}\Xi_m(\mathbf{p})\|_\infty \\ &\leq L_2 \|\boldsymbol{\alpha} - \boldsymbol{\alpha}'\|_1, \end{aligned} \tag{44}$$

where the last inequality is from **Lemma 1**. In addition, the left hand side of (42c) can be upper bounded as

$$\begin{aligned} &\|[\beta_1\Xi_1(\mathbf{p}) - \beta_1\Xi_1(\mathbf{p}'), \cdots, \beta_M\Xi_M(\mathbf{p}) - \beta_M\Xi_M(\mathbf{p}')]\|_\infty \\ &\leq \max_{m=1,\cdots,M} \beta_m \times \sup_{\mathbf{p}\in\mathcal{P}} \|\nabla_{\mathbf{p}}\Xi_m(\mathbf{p})\|_2 \times \|\mathbf{p} - \mathbf{p}'\|_2 \\ &\leq L_2 \|\mathbf{p} - \mathbf{p}'\|_1. \end{aligned} \tag{45}$$

Finally, since the left hand side of (42d) is zero, we immediately have that (42d) holds.

REFERENCES

[1] V. Vapnik, *The Nature of Statistical Learning Theory*. New York, NY, USA: Springer, 1998.

[2] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.

[3] M. Stolpe, "The Internet of Things: Opportunities and challenges for distributed data analysis," *ACM SIGKDD Explor. Newslett.*, vol. 18, no. 1, pp. 15–34, 2016.

[4] H. Li, K. Ota, and M. Dong, "Learning IoT in edge: Deep learning for the Internet of Things with edge computing," *IEEE Netw.*, vol. 32, no. 1, pp. 96–101, Jan. 2018.

[5] G. Zhu, D. Liu, Y. Du, C. You, J. Zhang, and K. Huang, "Toward an intelligent edge: Wireless communication meets machine learning," *IEEE Commun. Mag.*, vol. 58, no. 1, pp. 19–25, Jan. 2020.

[6] J. Park, S. Samarakoon, M. Bennis, and M. Debbah, "Wireless network intelligence at the edge," *Proc. IEEE*, vol. 107, no. 11, pp. 2204–2239, Nov. 2019.

[7] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

[8] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, Fort Lauderdale, FL, USA, Apr. 2017, pp. 1273–1282.

[9] S. Wang *et al.*, "Adaptive federated learning in resource constrained edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1205–1221, Jun. 2019.

[10] D. Gunduz, P. de Kerret, N. Sidiropoulos, D. Gesbert, C. Murthy, and M. van der Schaar, "Machine learning in the air," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2184–2199, Oct. 2019.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.

[11] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2224–2287, 3rd Quart., 2019.

[12] F. Tariq, M. Khandaker, K.-K. Wong, M. Imran, M. Bennis, and M. Debbah, "A speculative study on 6G," 2019, *arXiv:1902.06700*. [Online]. Available: http://arxiv.org/abs/1902.06700

[13] W. Saad, M. Bennis, and M. Chen, "A vision of 6G wireless systems: Applications, trends, technologies, and open research problems," *IEEE Netw.*, vol. 34, no. 3, pp. 134–142, May 2020.

[14] K. B. Letaief, W. Chen, Y. Shi, J. Zhang, and Y.-J.-A. Zhang, "The roadmap to 6G: AI empowered wireless networks," *IEEE Commun. Mag.*, vol. 57, no. 8, pp. 84–90, Aug. 2019.

[15] W. Yu, W. Rhee, S. Boyd, and J. M. Cioffi, "Iterative water-filling for Gaussian vector multiple-access channels," *IEEE Trans. Inf. Theory*, vol. 50, no. 1, pp. 145–152, Jan. 2004.

[16] M. Schubert and H. Boche, "Solution of the multiuser downlink beamforming problem with individual SINR constraints," *IEEE Trans. Veh. Technol.*, vol. 53, no. 1, pp. 18–28, Jan. 2004.

[17] H. Al-Shatri and T. Weber, "Achieving the maximum sum rate using D.C. programming in cellular networks," *IEEE Trans. Signal Process.*, vol. 60, no. 3, pp. 1331–1341, Mar. 2012.

[18] A. Beck, A. Nedić, A. Ozdaglar, and M. Teboulle, "An $O(1/k)$ gradient method for network resource allocation problems," *IEEE Trans. Control Netw. Syst.*, vol. 1, no. 1, pp. 64–73, Mar. 2014.

[19] S. Geman, E. Bienenstock, and R. Doursat, "Neural networks and the bias/variance dilemma," *Neural Comput.*, vol. 4, no. 1, pp. 1–58, Jan. 1992.

[20] P. Domingos, "A unified bias-variance decomposition and its applications," in *Proc. ICML*, Stanford, CA, USA, 2000, pp. 231–238.

[21] M. Johnson, P. Anderson, M. Dras, and M. Steedman, "Predicting classification error on large datasets from smaller pilot data," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, Melbourne, VIC, Australia, vol. 2, 2018, pp. 450–455.

[22] P. Kolachina, N. Cancedda, M. Dymetman, and S. Venkatapathy, "Prediction of learning curves in machine translation," in *Proc. 50th Annu. Meeting Assoc. Comput. Linguistics*, Jeju Island, South Korea, vol. 1, 2012, pp. 22–30.

[23] I. Y. Chen, F. D. Johansson, and D. Sontag, "Why is my classifier discriminatory?" in *Proc. Neural Inf. Process. Syst. (NIPS)*, Montréal, QC, Canada, 2018, pp. 3543–3554.

[24] H. S. Seung, H. Sompolinsky, and N. Tishby, "Statistical mechanics of learning from examples," *Phys. Rev. A, Gen. Phys.*, vol. 45, no. 8, pp. 6056–6091, Apr. 1992.

[25] D. Yarowsky, "Unsupervised word sense disambiguation rivaling supervised methods," in *Proc. 33rd Annu. meeting Assoc. Comput. Linguistics*, Stroudsburg, PA, USA, 1995, pp. 189–196.

[26] I. Radosavovic, P. Dollar, R. Girshick, G. Gkioxari, and K. He, "Data distillation: Towards omni-supervised learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4119–4128.

[27] *Performance Requirements for Narrowband IoT, LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Base Station (BS) Radio Transmission and Reception*, document TS 36.104 version 15.3.0 Release, 3GPP, vol. 15, Jul. 2018.

[28] Y. Xiao, Y. Jia, C. Liu, X. Cheng, J. Yu, and W. Lv, "Edge computing security: State of the art and challenges," *Proc. IEEE*, vol. 107, no. 8, pp. 1608–1631, Aug. 2019.

[29] K. Zheng, H. Meng, P. Chatzimisios, L. Lei, and X. Shen, "An SMDP-based resource allocation in vehicular cloud computing systems," *IEEE Trans. Ind. Electron.*, vol. 62, no. 12, pp. 7920–7928, Dec. 2015.

[30] L. Deng, "The MNIST database of handwritten digit images for machine learning research [best of the Web]," *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 141–142, Nov. 2012.

[31] *5G; NR; Physical Layer Procedures for Control*, version 15.5.0 Release 15, document TS 38.213, 3GPP, May 2019.

[32] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," 2019, *arXiv:1909.07972*. [Online]. Available: http://arxiv.org/abs/1909.07972

[33] B. Settles, "Active learning literature survey," Dept. Comput. Sci., Univ. Wisconsin-Madison, Madison, WI, USA, Tech. Rep. 1648, Jan. 2009.

[34] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Oct. 2011.

[35] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.

[36] R. Q. Charles, H. Su, M. Kaichun, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 652–660.

[37] B. R. Marks and G. P. Wright, "A general inner approximation algorithm for nonconvex mathematical programs," *Oper. Res.*, vol. 26, no. 4, pp. 681–683, Aug. 1978.

[38] T. Lipp and S. Boyd, "Variations and extension of the convex–concave procedure," *Optim. Eng.*, vol. 17, no. 2, pp. 263–287, Jun. 2016.

[39] Y. Sun, P. Babu, and D. P. Palomar, "Majorization-minimization algorithms in signal processing, communications, and machine learning," *IEEE Trans. Signal Process.*, vol. 65, no. 3, pp. 794–816, Feb. 2017.

[40] S. Wang, M. Xia, K. Huang, and Y.-C. Wu, "Wirelessly powered two-way communication with nonlinear energy harvesting model: Rate regions under fixed and mobile relay," *IEEE Trans. Wireless Commun.*, vol. 16, no. 12, pp. 8190–8204, Dec. 2017.

[41] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[42] A. Ben-Tal and A. Nemirovski, *Lectures on Modern Convex Optimization* (MPS/SIAM Series on Optimizations). Philadelphia, PA, USA: SIAM, 2013.

[43] L. Lu, G. Y. Li, A. L. Swindlehurst, A. Ashikhmin, and R. Zhang, "An overview of massive MIMO: Benefits and challenges," *IEEE J. Sel. Topics Signal Process.*, vol. 8, no. 5, pp. 742–758, Oct. 2014.

[44] H. Q. Ngo, E. G. Larsson, and T. L. Marzetta, "Energy and spectral efficiency of very large multiuser MIMO systems," *IEEE Trans. Commun.*, vol. 61, no. 4, pp. 1436–1449, Apr. 2013.

[45] L. Liu and W. Yu, "Massive connectivity with massive MIMO—Part I: Device activity detection and channel estimation," *IEEE Trans. Signal Process.*, vol. 66, no. 11, pp. 2933–2946, Jun. 2018.

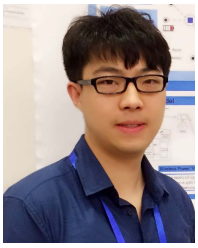
[46] K. E. Atkinson, *An Introduction to Numerical Analysis*, 2nd ed. New York, NY, USA: Wiley, 1989.

[47] A. Nemirovski, "Prox-method with rate of convergence $\mathcal{O}(\infty/\mathcal{T})$ for variational inequalities with Lipschitz continuous monotone operators and smooth convex-concave saddle point problems," *SIAM J. Optim.*, vol. 15, no. 1, pp. 229–251, Jan. 2004.

[48] S. Bubeck, "Convex optimization: Algorithms and complexity," *Found. Trends Mach. Learn.*, vol. 8, nos. 3–4, pp. 231–357, 2015.

[49] A. Konar and N. D. Sidiropoulos, "Fast approximation algorithms for a class of non-convex QCQP problems using first-order methods," *IEEE Trans. Signal Process.*, vol. 65, no. 13, pp. 3494–3509, Jul. 2017.

[50] S. Yu, B. Dab, Z. Movahedi, R. Langar, and L. Wang, "A socially-aware hybrid computation offloading framework for multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 19, no. 6, pp. 1247–1259, Jun. 2020.



Shuai Wang (Member, IEEE) received the Ph.D. degree in electrical and electronic engineering from The University of Hong Kong (HKU) in 2018. From 2018 to 2019, he was a Post-Doctoral Fellow and a Lecturer with HKU. He is currently a Research Assistant Professor with the Southern University of Science and Technology (SUSTech). He has authored more than 20 articles in top-level IEEE journals and flagship international conferences. His research interests include wireless communications, unmanned systems, and machine learning. He received the Best Paper Award from the IEEE ICC'2020. He was recognized as an Exemplary Reviewer for the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS in 2018 and the IEEE WIRELESS COMMUNICATIONS LETTERS in 2019.



Yik-Chung Wu (Senior Member, IEEE) received the B.Eng. degree in EEE and the M.Phil. degree from The University of Hong Kong (HKU) in 1998 and 2001, respectively, and the Ph.D. degree from Texas A&M University, College Station, TX, USA, in 2005. From 2005 to 2006, he was with the Thomson Corporate Research, Princeton, NJ, USA, as a member of Technical Staff. Since 2006, he has been with HKU as an Associate Professor. He was a Visiting Scholar with Princeton University, in summers of 2015 and 2017. His research interests are in general areas of signal processing, machine learning, and communication systems. He received the Croucher Foundation Scholarship from his Ph.D. degree, in 2002. He has served as an Editor for the IEEE COMMUNICATIONS LETTERS and the IEEE TRANSACTIONS ON COMMUNICATIONS. He is currently an Editor of the JOURNAL OF COMMUNICATIONS AND NETWORKS.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.



Minghua Xia (Member, IEEE) received the Ph.D. degree in telecommunications and information systems from Sun Yat-sen University, Guangzhou, China, in 2007.

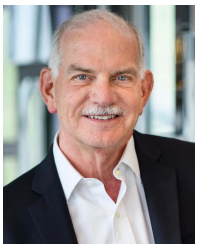
From 2007 to 2009, he was with the Electronics and Telecommunications Research Institute (ETRI), South Korea, the Beijing Research and Development Center, Beijing, China, where he worked as a member and then as a Senior Member of engineering staff. From 2010 to 2014, he was a Post-Doctoral Fellow in sequence with The University of Hong Kong, Hong Kong, China, the King Abdullah University of Science and Technology, Jeddah, Saudi Arabia, and the Institut National de la Recherche Scientifique (INRS), University of Quebec, Montreal, QC, Canada. Since 2015, he has been a Professor with Sun Yat-sen University. Since 2019, he has also been an Adjunct Professor with the Southern Marine Science and Engineering Guangdong Laboratory, Zhuhai. His research interests are in the general areas of wireless communications and signal processing.

Dr. Xia received the Professional Award at the IEEE TENCON, held in Macau, in 2015. He has served as the TPC Symposium Chair of the IEEE ICC'2019. He currently serves as a TPC Symposium Chair for the IEEE ICC'2021, and an Associate Editor for the IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING and *IET Smart Cities*. He was recognized as an Exemplary Reviewer of the IEEE TRANSACTIONS ON COMMUNICATIONS in 2014, the IEEE COMMUNICATIONS LETTERS in 2014, and the IEEE WIRELESS COMMUNICATIONS LETTERS in 2014 and 2015.



Rui Wang (Member, IEEE) received the B.S. degree from the University of Science and Technology of China in 2004 and the Ph.D. degree in wireless communications from The Hong Kong University of Science and Technology in 2008.

From 2009 to 2012, he was a Senior Research Engineer with Huawei Technologies, Company, Ltd. Since 2012, he has been with the Southern University of Science and Technology of China as an Associate Professor. He has research experience in both academia and industry. He has authored more than 30 articles in top-level IEEE journals and flagship international conferences, especially in the area of wireless radio resource optimization and interference management. He has contributed more than 20 U.S. patent applications and more than 30 Chinese patent applications (20 of them have been granted). He also involved in the development of interference mitigation technology for 5G systems.



H. Vincent Poor (Life Fellow, IEEE) received the Ph.D. degree in EECS from Princeton University in 1977.

From 1977 to 1990, he was on the faculty of the University of Illinois at Urbana–Champaign. Since 1990, he has been on the faculty at Princeton, where he is currently the Michael Henry Strater University Professor of electrical engineering. From 2006 to 2016, he has served as the Dean of the Princeton's School of Engineering and Applied Science. He has also held visiting appointments at several other universities, including most recently at Berkeley and Cambridge. His research interests are in the areas of information theory, signal processing and machine learning, and their applications in wireless networks, energy systems, and related fields. Among his publications in these areas is the recent book *Multiple Access Techniques for 5G Wireless Networks and Beyond* (Springer, 2019). He is a member of the National Academy of Engineering and the National Academy of Sciences, and is a Foreign Member of the Chinese Academy of Sciences, the Royal Society, and other national and international academies. Recent recognition of his work includes the 2017 IEEE Alexander Graham Bell Medal and the D.Eng. *honoris causa* from the University of Waterloo awarded in 2019.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on October 19,2026 at 18:33:55 UTC from IEEE Xplore. Restrictions apply.